Lab Tasks

Name: Muhammad Faisal

Roll no: 2022F-BCS-152

Section: A

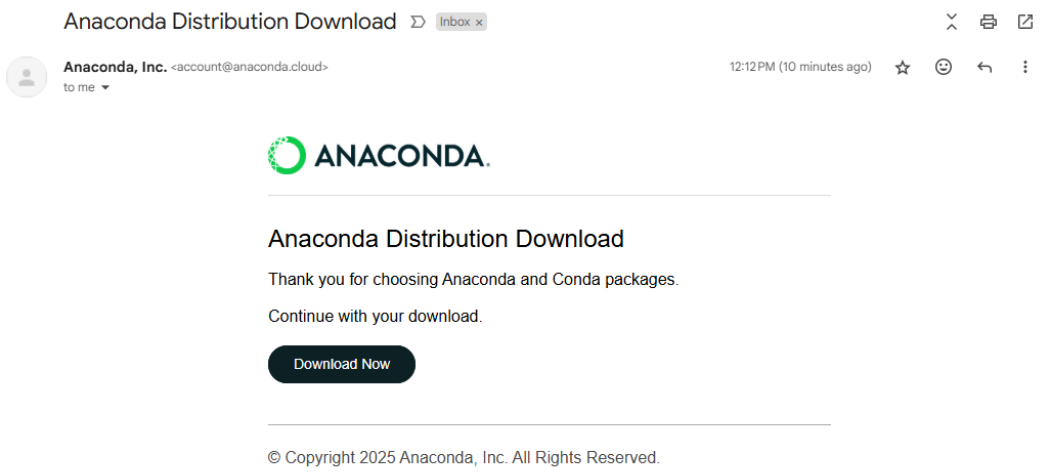
Instructor Name: Miss Fozia Khan

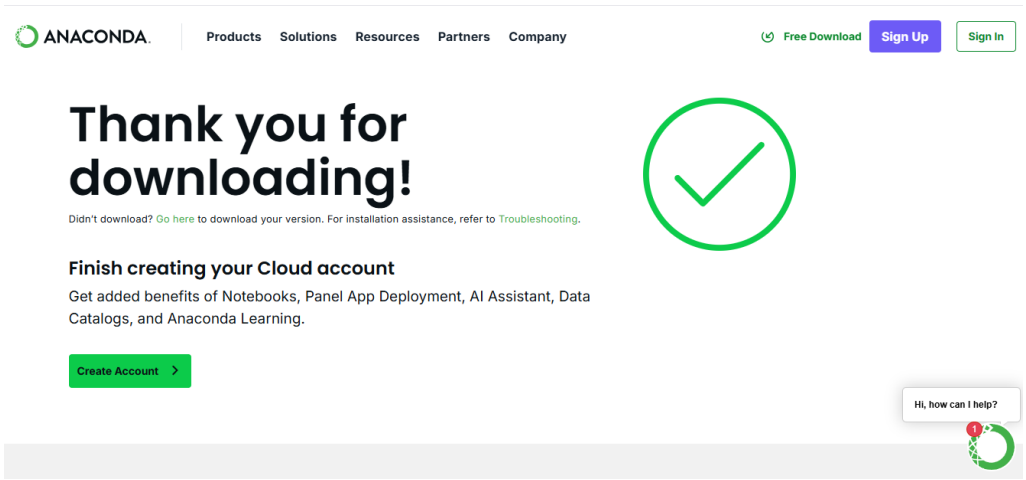
Course: Artificial Intelligence Lab (CS - 325L)

Program: (BS) - Computer Science

Lab #00

Tasks 01 : Install the anaconda Nav.





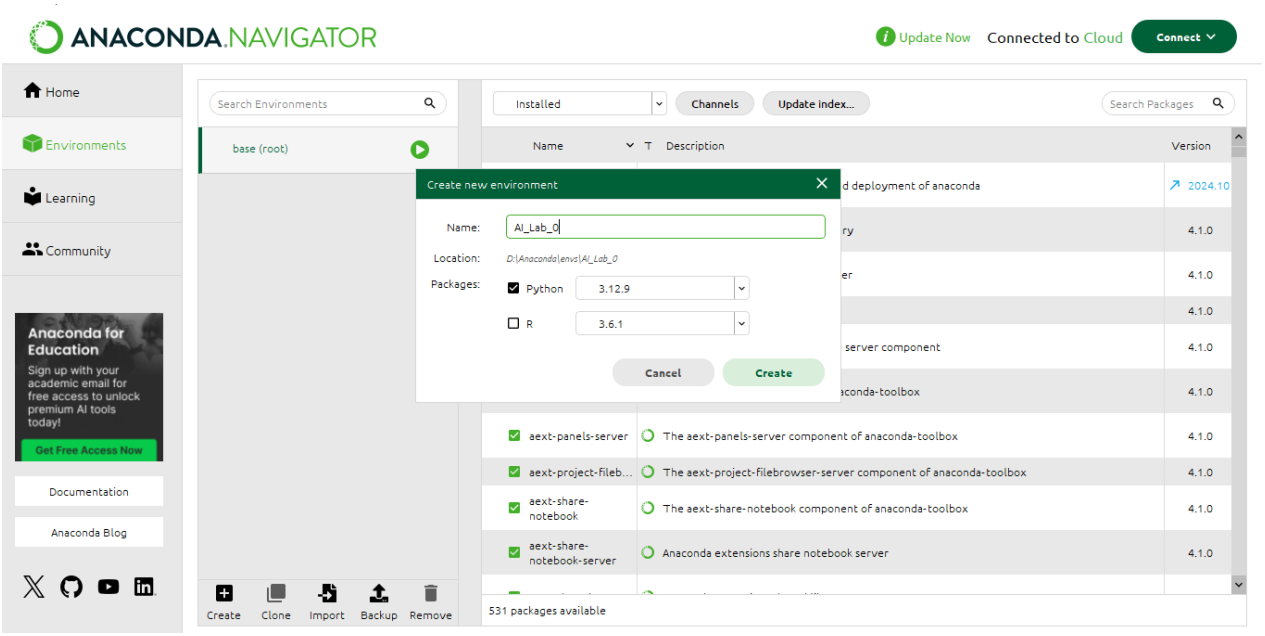
Task 02 : Answer the following.

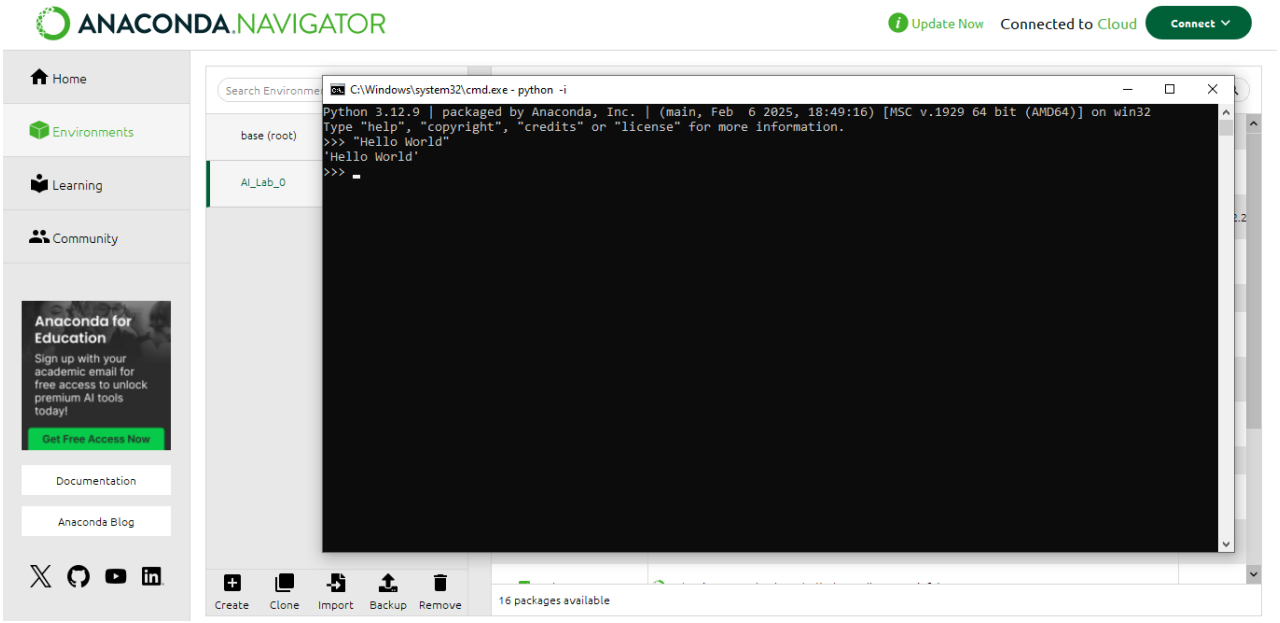
1. **What is Environment?**
   In computing, an **environment** refers to a setup or space where software or applications run. It includes the configuration, tools, libraries, and system settings needed for development, testing, or deployment.

   Examples:
   - Development environment (for coding and testing)
   - Production environment (for running the actual app used by users)
   - Virtual environments (isolated setups for specific projects)

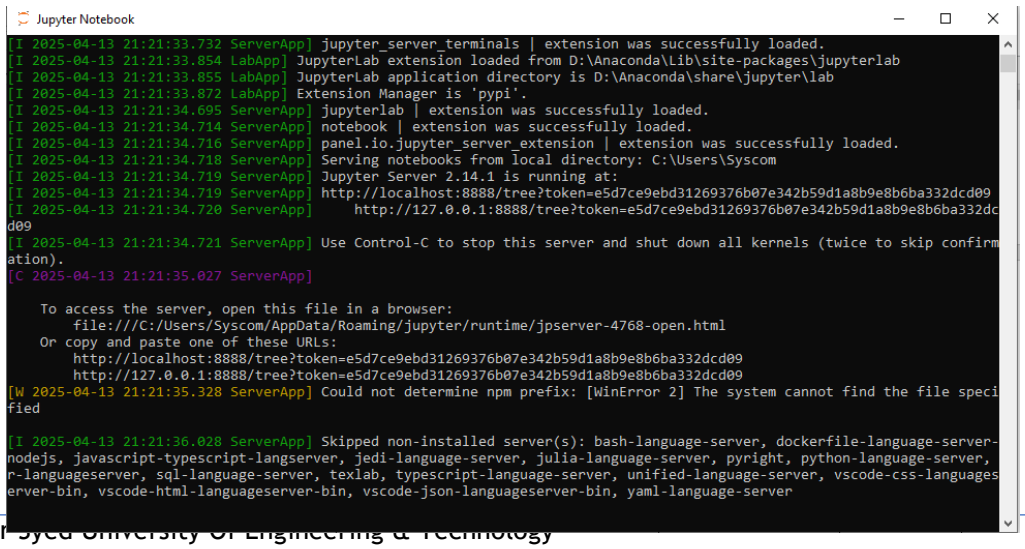
2. **Why do we make an Environment?**
   We make environments to:
   - Keep things organized (separate development from production)
   - Avoid conflicts between software packages or dependencies
   - Test safely without affecting live applications
   - Control versions of tools or libraries per project

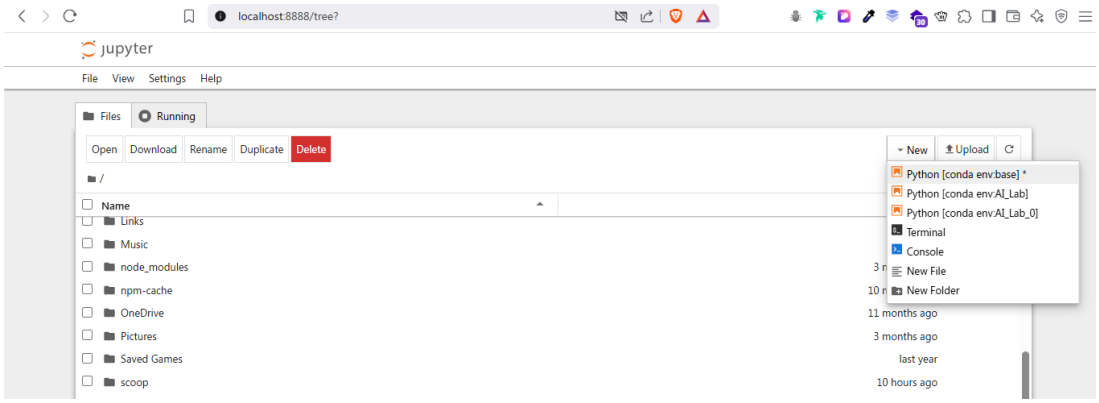
3. How do we make an Environment? Create one.

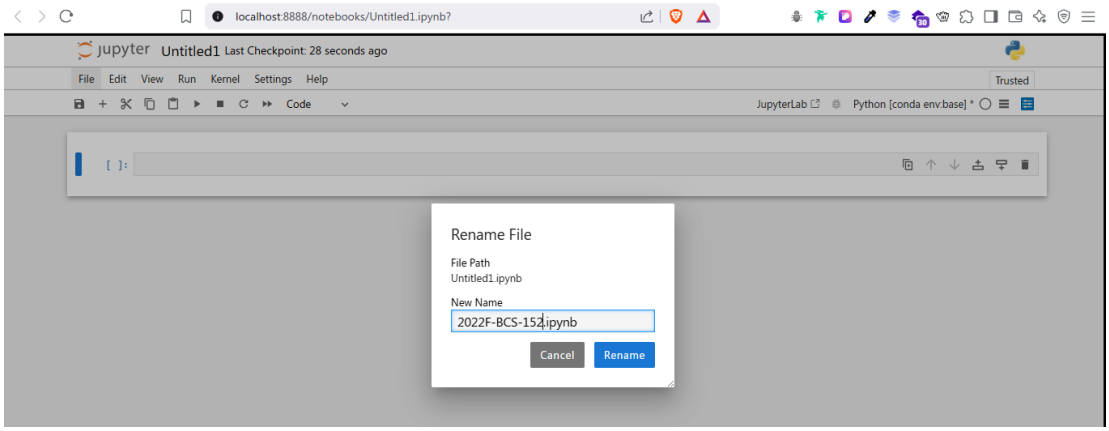




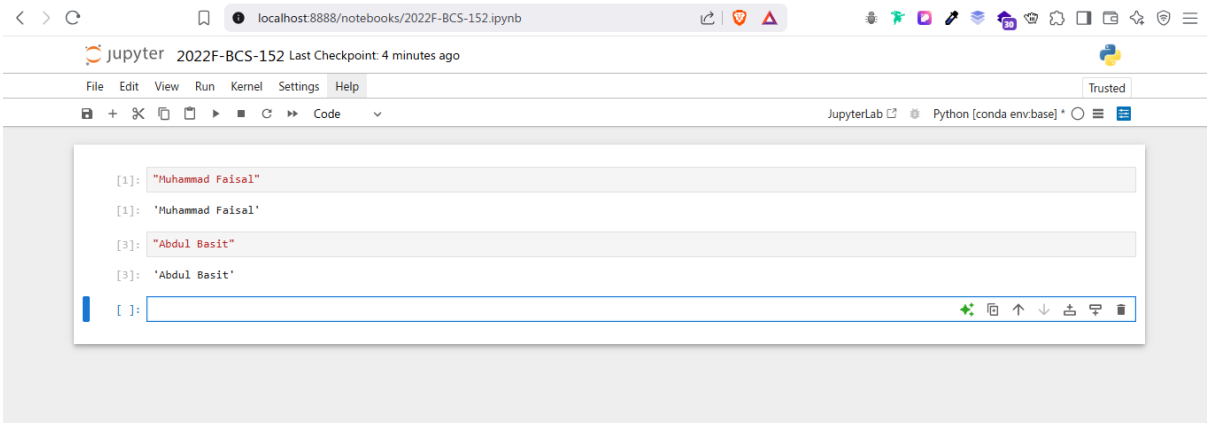
Task 03: Create an empty file on Jupyter notebook and rename it using your Roll no. [Show all the steps].







Task 04: Print your name along with your fathers on the notebook created earlier.



Lab #01

Activity- 1:

Write a script that take user input for a number then adds 3 to that number. Then multiplies the result by 2, subtract 4, then again adds 3, then print the result.

```
num = int(input("Enter a number:"))
result=(((num+3)*2)-4)+3
print("Final result:",result)
```

```
Enter a number: 2
Final result: 9
```

Activity- 2:

Write a script that takes input as radius then calculate area of circle. (Hint: A= πr^2).

```
import math
radius=float(input("Enter the radius:"))
area=math.pi*radius**2
print("Area of the circle:",area)
```

```
Enter the radius: 3.2
Area of the circle: 32.169908772759484
```

Activity- 3:

Write a Python script that asks users for their favourite color. Create the following output (assuming blue is the chosen color) (hint: use „+" and „*")

blueblueblueblueblueblueblueblueblue
blue blue
blueblueblueblueblueblueblueblueblue

```
color=input("Enter your favourite color:")
print(color*10)
print(color+ "                                  " +color)
print(color*10)
```

```
Enter your favourite color: blue
blueblueblueblueblueblueblueblueblue
blue                                blue
blueblueblueblueblueblueblueblueblue
```

Activity- 4:

Store a person"s name, and include some „*" characters at the beginning and end of the name. Print the name once, so the „*" around the name is displayed. Then print the name using each of the three stripping functions, lstrip(), rstrip(), and strip().

```
name= "***Faisal***"
print("Original:",name)
print("lstrip():",name.lstrip('*'))
print("rstrip():",name.rstrip('*'))
print("strip():",name.strip('*'))
```

```
Original: ***Faisal***
lstrip(): Faisal***
rstrip(): ***Faisal
strip(): Faisal
```

Activity- 5:
Write a function called absolute_num() that accepts one parameter, num. The function should return only positive value, and apply condition on it. This function returns the absolute value of the entered number.

```
def absolute_num(num):
    if num<0:
        return -num
    else:
        return num

print(absolute_num(int(input("Enter a number:"))))
```

```
Enter a number: -5
5
```

Activity- 6:
Write a function called describe_city( ) that accepts the name of a city and its country. The function should print a simple sentence, such as Karachi is in Pakistan. Give the parameter for the country a default value. Call your function for three different cities, at least one of which is in the default country.

```
def describe_city(city,country="Pakistan"):
    print(f"{city} is in {country}.")

describe_city("Karachi")
describe_city("Lahore")
describe_city("Tokyo","Japan")
```

```
Karachi is in Pakistan.
Lahore is in Pakistan.
Tokyo is in Japan.
```

Activity- 7:
Write a python script that take a user input and to create the multiplication table (from 1 to 10) of that number.

```
num=int(input("Enter a number for multipication table:"))
for i in range(1,11):
    print(f"{num}x{i} = {num*i}")
```

```
Enter a number for multipication table: 3
3x1 = 3
3x2 = 6
3x3 = 9
3x4 = 12
3x5 = 15
3x6 = 18
3x7 = 21
3x8 = 24
3x9 = 27
3x10 = 30
```

Activity- 8:
Write a Python program that prints all the numbers from 0 to 6 except 3 and 6.
Note: Use 'continue' statement.

```
for i in range(7):
    if i == 3 or i==6:
       continue
    print(i)
```

```
0
1
2
4
5
```

Activity- 9:
Stages of Life: Write an if-elif-else chain that determines a person‟s stage of life. Set a value for the variable age, and then:

- **If the person is less than 2 years old, print a message that the person is a baby.**
- **If the person is at least 4 years old but less than 13, print a message that the person is a kid.**
- **If the person is at least 13 years old but less than 20, print a message that the person is a teenager.**
- **If the person is at least 20 years old but less than 65, print a message that the person is an adult.**
- **If the person is age 65 or older, print a message that the person is an old.**

```
age = int(input("Enter your age: "))

if age < 2:
    print("The person is a baby.")
elif age >= 4 and age < 13:
    print("The person is a kid.")
elif age >= 13 and age < 20:
    print("The person is a teenager.")
elif age >= 20 and age < 65:
    print("The person is an adult.")
elif age >= 65:
    print("The person is old.")
else:
    print("The person is a toddler.")
```

```
Enter your age:  22
The person is an adult.
```

Lab #02

Activity- 1:

Write a Python script that uses a dictionary to store information about a person you know. Store their first name, last name, age, and the city in which they live. You should have keys such as first_name, last_name, age, and city. Print each piece of information stored in your dictionary.

```
person = {
    'first_name': 'Ali',
    'last_name': 'Khan',
    'age': 25,
    'city': 'Karachi'
}
print("Activity 1:")
for key, value in person.items():
    print(f"{key.capitalize()}: {value}")
print()
```

```
Activity 1:
First_name: Ali
Last_name: Khan
Age: 25
City: Karachi
```

Activity- 2:

Write a function that accepts a dictionary as an argument. If the dictionary contains replicate values, return an empty dictionary, otherwise, return a new dictionary whose values are now the keys and whose keys are the values.

```
def swap_dict(d):
    if len(set(d.values())) != len(d.values()):
        return {}
    return {v: k for k, v in d.items()}

print("Activity 2:")
sample_dict = {'a': 1, 'b': 2, 'c': 3}
print(swap_dict(sample_dict))
sample_dict_with_duplicates = {'a': 1, 'b': 2, 'c': 1}
print(swap_dict(sample_dict_with_duplicates))
print()
```

```
Activity 2:
{1: 'a', 2: 'b', 3: 'c'}
{}
```

Activity- 3:

A buffet-style restaurant offers only five basic foods. Think of five simple foods, and store them in a tuple.

- **Use a for loop to print each food the restaurant offers.**
- **Try to modify one of the items, and make sure that Python rejects the change.**

```
print("Activity 3:")
foods = ('Pizza', 'Burger', 'Pasta', 'Fries', 'Salad')
for food in foods:
    print(food)
foods[0] = 'Sushi'
print()
```

```
Activity 3:
Pizza
Burger
Pasta
Fries
Salad
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 5
      3 for food in foods:
      4     print(food)
----> 5 foods[0] = 'Sushi'
      6 print()

TypeError: 'tuple' object does not support item assignment
```

Activity- 4:

If you could invite anyone to dinner. Make a list that includes at least three people you'd like to invite to dinner. Then use your list to print a message to each person, inviting them to dinner.

```
print("Activity 4:")
guests = ['Hamza', 'Sara', 'Ahmed']
for guest in guests:
    print(f"Dear {guest}, you're invited to dinner at my place!")
print()
```

```
Activity 4:
Dear Hamza, you're invited to dinner at my place!
Dear Sara, you're invited to dinner at my place!
Dear Ahmed, you're invited to dinner at my place!
```

Activity- 5:

Changing Guest List: You just heard that one of your guests can't make the dinner, so you need to send out a new set of invitations. You'll have to think of someone else to invite.

- **Modify your list, replacing the name of the guest who can't make it with the name of the new person you are inviting.**
- **Print a second set of invitation messages, one for each person who is still in your list.**

```
print(f"{guests[1]} can't make it to dinner.")
guests[1] = 'Bilal'
for guest in guests:
    print(f"Dear {guest}, you're still invited to dinner at my place!")
print()
```

```
Activity 5:
Fatima can't make it to dinner.
Dear Hamza, you're still invited to dinner at my place!
Dear Bilal, you're still invited to dinner at my place!
Dear Ahmed, you're still invited to dinner at my place!
```

Activity- 6:
Write a program to read 6 numbers and create a dictionary having keys EVEN and ODD.
Dictionary's value should be stored in list. Your dictionary should be like: {'EVEN':[8,10,64], 'ODD':[1,5,9]}

```
print("Activity 6:")
numbers = []
print("Enter 6 numbers:")
for _ in range(6):
    numbers.append(int(input()))
result = {'EVEN': [], 'ODD': []}
for n in numbers:
    if n % 2 == 0:
        result['EVEN'].append(n)
    else:
        result['ODD'].append(n)
print(result)
print()
```

```
Activity 6:
Enter 6 numbers:
 1
 2
 6
 7
 3
 4
{'EVEN': [2, 6, 4], 'ODD': [1, 7, 3]}
```

Activity- 7:
Write a definition of a method count_now(places) to find and display those place names, in which there are more than 5 characters.

```
def count_now(places):
    print("Activity 7:")
    for place in places:
        if len(place) > 5:
            print(place)

count_now(['London', 'Tokyo', 'Istanbul', 'Karachi', 'New York'])
print()
```

```
Activity 7:
London
Istanbul
Karachi
New York
```

Activity- 8:
Write the following 2 functions: def ComputeOddSum(num): def ComputeEvenSum(num):

- **The function ComputeOddSum find the sum of all odd numbers less than num.**
- **The function ComputeEvenSum find the sum of all even numbers less than num.**

```python
def ComputeOddSum(num):
    return sum(i for i in range(num) if i % 2 != 0)

def ComputeEvenSum(num):
    return sum(i for i in range(num) if i % 2 == 0)

print("Activity 8:")
print("Odd sum under 10:", ComputeOddSum(10))
print("Even sum under 10:", ComputeEvenSum(10))
print()
```

```
Activity 8:
Odd sum under 10: 25
Even sum under 10: 20
```

Activity- 9:

Write a recursive function to get sum of all number from 1 up to give number. Example N = 5 Result must be sum (1+2+3+4+5) =15.

```python
def recursive_sum(n):
    if n <= 1:
        return n
    return n + recursive_sum(n - 1)

print("Activity 9:")
print("Sum up to 5:", recursive_sum(5))
```

```
Activity 9:
Sum up to 5: 15
```

Lab #03

Class Employee
***Employee*: Parameterized constructor.**
***getter & setter:* Create the get and set method to each attribute. *display:* Displays Employee ID and Name in the following format: ID: 1234 – Name: XYZ - Salary: 70000.00**

Class Faculty
***Faculty*: Parameterized constructor.**
***getter & setter:* Create the get and set method to each attribute. *display:* Shows the Faculty information in the following format: ID: 1234 – Name: XYZ – Degree: PhD - Salary: 70000.00 salary: Calculate salary based on the following formula :**
Salary= basicSalary + teachingHours * 1000

Class Staff
***Staff:* Parameterized constructor.**
***getter & setter:* Create the get and set method to each attribute. *display:* Shows the Staff information in the following format:**
ID: 1234 – Name: XYZ – JobTitle: Registrar –Salary: 70000.00

salary: Calculate salary based on the following formula:
Salary= basicSalary
Unless workingHours > 8, Then
Salary=basicSalary+ (basicSalary *25 /100)

Once you are done implementing the classes, you need to test your work using the class with main method to do the following:

- **Create an instance of class Employee, an instance of class Faculty and an instance of class Staff with proper values for the attributes.**
- **Display the information for each instance using display() method.**
- **Show the salary for Faculty and Staff.**

Run example:
ID: 43221 - Name: Ali - Salary: 70000.0 PKR
ID: 71245 - Name: Majed - Degree: PhD - Teaching hours: 15 - Salary:182000.0 PKR
ID: 81234 - Name: Nasser – Job Title: Registrar – Working hours: 10 - Salary:125000.0 PKR

```
class Employee:
   def __init__(self, emp_id, name, salary):
      self.emp_id = emp_id
      self.name = name
      self.salary = salary

   def get_id(self): return self.emp_id
   def set_id(self, emp_id): self.emp_id = emp_id

   def get_name(self): return self.name
   def set_name(self, name): self.name = name

   def get_salary(self): return self.salary
   def set_salary(self, salary): self.salary = salary

   def display(self):
      print(f"ID: {self.emp_id} - Name: {self.name} - Salary: {self.salary:.1f} PKR")
```

```
class Faculty(Employee):
    def __init__(self, emp_id, name, basic_salary, degree, teaching_hours):
        self.degree = degree
        self.teaching_hours = teaching_hours
        salary = basic_salary + teaching_hours * 1000
        super().__init__(emp_id, name, salary)

    def get_degree(self): return self.degree
    def set_degree(self, degree): self.degree = degree

    def get_teaching_hours(self): return self.teaching_hours
    def set_teaching_hours(self, hours): self.teaching_hours = hours

    def display(self):
        print(f"ID: {self.emp_id} - Name: {self.name} - Degree: {self.degree} - Teaching hours: {self.teaching_hours} - Salary:
{self.salary:.1f} PKR")

class Staff(Employee):
    def __init__(self, emp_id, name, basic_salary, job_title, working_hours):
        self.job_title = job_title
        self.working_hours = working_hours
        if working_hours > 8:
            salary = basic_salary + (basic_salary * 0.25)
        else:
            salary = basic_salary
        super().__init__(emp_id, name, salary)

    def get_job_title(self): return self.job_title
    def set_job_title(self, title): self.job_title = title

    def get_working_hours(self): return self.working_hours
    def set_working_hours(self, hours): self.working_hours = hours

    def display(self):
        print(f"ID: {self.emp_id} - Name: {self.name} - Job Title: {self.job_title} - Working hours: {self.working_hours} - Salary:
{self.salary:.1f} PKR")

if __name__ == "__main__":
    emp = Employee(43221, "Ali", 70000)
    faculty = Faculty(71245, "Majed", 32000, "PhD", 15)
    staff = Staff(81234, "Nasser", 100000, "Registrar", 10)

    emp.display()
    faculty.display()
    staff.display()
```

```
ID: 43221 - Name: Ali - Salary: 70000.0 PKR
ID: 71245 - Name: Majed - Degree: PhD - Teaching hours: 15 - Salary: 47000.0 PKR
ID: 81234 - Name: Nasser - Job Title: Registrar - Working hours: 10 - Salary: 125000.0 PKR
```

Lab #04

Activity- 1:

In contrast to the standard FIFO implementation of Queue, the LifoQueue uses last-in, first-out ordering (normally associated with a stack data structure). Implement LIFO queue using queue module.

```
from queue import LifoQueue

# Create a LIFO queue
stack = LifoQueue()

# Push elements into the stack
stack.put(10)
stack.put(20)
stack.put(30)

# Pop elements from the stack (Last In First Out)
while not stack.empty():
    print(stack.get())
```

```
30
20
10
```

Activity- 2:

We have an array of 5 elements: [4, 8, 1, 7, 3] and we have to insert all the elements in the maxpriority queue. First as the priority queue is empty, so 4 will be inserted initially. Now when 8 will be inserted it will move to front as 8 is greater than 4. While inserting 1, as it is the current minimum element in the priority queue, it will remain in the back of priority queue. Now 7 will be inserted between 8 and 4 as 7 is smaller than 8. Now 3 will be inserted before 1 as it is the 2nd minimum element in the priority queue. All the steps are represented in the diagram below:


```
elements = [4, 8, 1, 7, 3]
priority_queue = []

for num in elements:
    # Insert in descending order
    inserted = False
    for i in range(len(priority_queue)):
        if num > priority_queue[i]:
            priority_queue.insert(i, num)
            inserted = True
            break
    if not inserted:
        priority_queue.append(num)
    print(priority_queue)
```

```
[4]
[8, 4]
[8, 4, 1]
[8, 7, 4, 1]
[8, 7, 4, 3, 1]
```

Activity- 3:

Consider a simple priority queue implementation for scheduling the presentations of students based on their roll number. Here roll number decides the priority of the student to present. Since it is a min-heap, roll number 1 is considered to be of the highest priority.

```
import heapq

# (roll_no, student_name)
students = [(3, "Ali"), (1, "Sara"), (4, "John"), (2, "Zara")]

heapq.heapify(students)

while students:
    roll_no, name = heapq.heappop(students)
    print(f"Roll No: {roll_no}, Name: {name}")
```

```
Roll No: 1, Name: Sara
Roll No: 2, Name: Zara
Roll No: 3, Name: Ali
Roll No: 4, Name: John
```

Activity- 4:

Initialize a heap with student marks and their names.

- **Print the heap.**
- **Print the name of students from highest to lowest marks scored by students.**

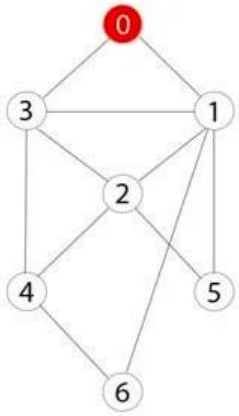
```
import heapq

# We use (-marks, name) to simulate max-heap
marks = [("Ali", 76), ("Sara", 90), ("John", 68), ("Zara", 85)]
max_heap = [(-score, name) for name, score in marks]
heapq.heapify(max_heap)

# Print heap
print("Heap:", max_heap)

# Print students from highest to lowest
while max_heap:
    score, name = heapq.heappop(max_heap)
    print(f"{name}: {-score}")
```

```
Heap: [(-90, 'Sara'), (-85, 'Zara'), (-68, 'John'), (-76, 'Ali')]
Sara: 90
Zara: 85
Ali: 76
John: 68
```

Lab #05

Activity- 1:

Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS algorithm, assume we always choose the letter closest to the beginning of the alphabet first. A connected graph with 7 nodes and 7 edges. The edges are undirected and unweight. Distance between two nodes will be measured based on the number of edges separating two vertices. Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors.



Define function name _connected_component', this function keep track of all the visited nodes with BFS, is as simple as implementing the steps of the algorithm and assign _queue' variable already has a node to be checked, i.e., the starting vertex that is used as an entry point to explore the graph. The next step is to implement a loop that keeps cycling until queue is empty. At each iteration of the loop, a node is checked. If this wasn't visited already, its neighbors are added to queue. Once the loop is exited, the function (connected_component) returns all of the visited nodes.

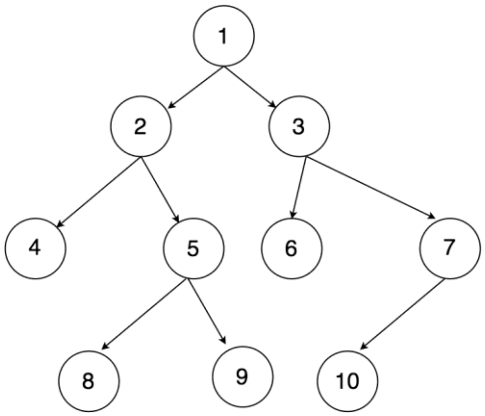
```
from collections import deque

def connected_component(graph, start):
    visited = []
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.append(node)
            # Get neighbors and sort them alphabetically/numerically to ensure correct order
            neighbors = sorted(graph[node])
            for neighbor in neighbors:
                if neighbor not in visited:
                    queue.append(neighbor)
    return visited

# Updated graph based on the image
graph = {
    0: [1, 2, 3],
    1: [0, 5],
    2: [0, 4, 5, 6],
    3: [0, 4],
    4: [2, 3],
    5: [1, 2],
    6: [2]
}

# Starting node for BFS
```

```
start_node = 0

# Get the connected component starting from '0'
component = connected_component(graph, start_node)
print("Connected component starting from", start_node, ":", component)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Connected component starting from 0 : [0, 1, 2, 3, 5, 4, 6]
```

Activity- 2:

Apply Breadth First Search on following graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.




```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
```

```
}

# Initial and goal states
start_node = 1
goal_node = 10

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("Open List:", list(open_list))
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Open List: []
Closed List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Activity- 3:

Repeat Activity-2, apply BFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Take user input for initial and goal states
start_node = int(input("Enter the initial state: "))
```

```
goal_node = int(input("Enter the goal state: "))

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("Open List:", list(open_list))
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 5
Open List: [6, 7]
Closed List: [1, 2, 3, 4, 5]
```

Activity- 4:

Apply Depth First Search on Activity-2 graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found
```

```
# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Initial and goal states
start_node = 1
goal_node = 10

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
DFS Open List: []
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6, 7, 10]
```

Activity- 5:

Repeat Activity-4, apply DFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

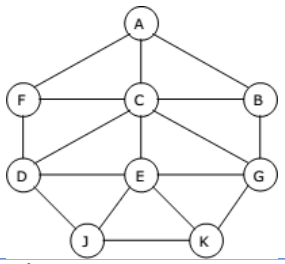
```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)
```

```
    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    1: [2, 3],
    2: [4, 5],
    3: [6, 7],
    4: [],
    5: [8, 9],
    6: [],
    7: [10],
    8: [],
    9: [],
    10: []
}

# Take user input for initial and goal states
start_node = int(input("Enter the initial state: "))
goal_node = int(input("Enter the goal state: "))

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 6
DFS Open List: [7]
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6]
```

Activity- 6:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Breadth First Search on following graph considering the initial state is A and final state is K. Also evaluate it manually.



```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Initial and goal states
start_node = 'A'
goal_node = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("BFS Open List:", list(open_list))
print("BFS Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
```

Activity- 7:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Depth First Search on Activity-6 graph considering the initial state is A and final state is K. Also evaluate it manually.

```
from collections import deque

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
```

```
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

# Graph structure based on the image
graph = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Initial and goal states
start_node = 'A'
goal_node = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("BFS Open List:", list(open_list))
```

```
print("BFS Closed List:", closed_list)

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

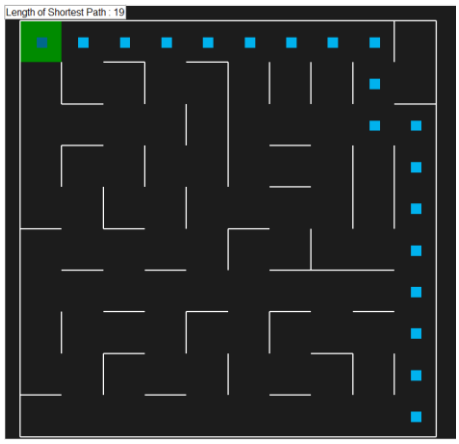
```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
DFS Open List: ['F', 'C']
DFS Closed List: ['A', 'B', 'G', 'E', 'J', 'D', 'K']
```

Activity 08:
You need to implement DFS and BFS both using "Pygame" and "Pyamaze" library, generate the maze of (20,20) and find the path.

Implementing BFS using Pyamaze:

```
from pyamaze import maze,agent,textLabel

def BFS(m):
    start=(m.rows,m.cols)
    frontier=[start]
    explored=[start]
    bfsPath={}
    while len(frontier)>0:
        currCell=frontier.pop(0)
        if currCell==(1,1):
            break
        for d in 'ESNW':
            if m.maze_map[currCell][d]==True:
                if d=='E':
                    childCell=(currCell[0],currCell[1]+1)
                elif d=='W':
                    childCell=(currCell[0],currCell[1]-1)
                elif d=='N':
                    childCell=(currCell[0]-1,currCell[1])
                elif d=='S':
                    childCell=(currCell[0]+1,currCell[1])
                if childCell in explored:
                    continue
                frontier.append(childCell)
                explored.append(childCell)
                bfsPath[childCell]=currCell
    fwdPath={}
    cell=(1,1)
    while cell!=start:
        fwdPath[bfsPath[cell]]=cell
        cell=bfsPath[cell]
    return fwdPath

if __name__=='__main__':
    m=maze(10  ,10)
    m.CreateMaze(loopPercent=100)
```

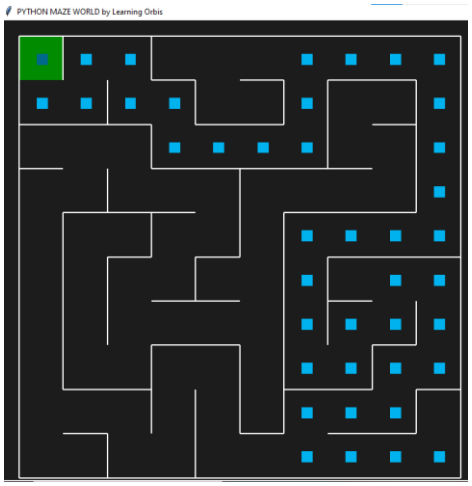
```
path=BFS(m)

a=agent(m,footprints=True)
m.tracePath({a:path})
l=textLabel(m,'Length of Shortest Path',len(path)+1)
print('Length of Shortest Path',len(path)+1)
print(m.maze_map)
m.run()
```



Implementing DFS using Pyamaze:

```
from pyamaze import maze,agent
def DFS(m):
    start=(m.rows,m.cols)
    explored=[start]
    frontier=[start]
    dfsPath={}
    while len(frontier)>0:
        currCell=frontier.pop()
        if currCell==(1,1):
            break
        for d in 'ESNW':
            if m.maze_map[currCell][d]==True:
                if d=='E':
                    childCell=(currCell[0],currCell[1]+1)
                elif d=='W':
                    childCell=(currCell[0],currCell[1]-1)
                elif d=='S':
                    childCell=(currCell[0]+1,currCell[1])
                elif d=='N':
                    childCell=(currCell[0]-1,currCell[1])
                if childCell in explored:
                    continue
                explored.append(childCell)
                frontier.append(childCell)
                dfsPath[childCell]=currCell
    fwdPath={}
    cell=(1,1)
    while cell!=start:
        fwdPath[dfsPath[cell]]=cell
        cell=dfsPath[cell]
    return fwdPath
```

```
if __name__=='__main__':
    m=maze(10 ,10)  #Setting maze 10 X 10
    m.CreateMaze()          # Now creating maze of soze 10 x 10
    path=DFS(m)    #calling method DFS which is returning path
    a=agent(m,footprints=True) #player declaration
    m.tracePath({a:path})       #tracing path by agent

   # print({a:path})
    m.run()   #Execution
```

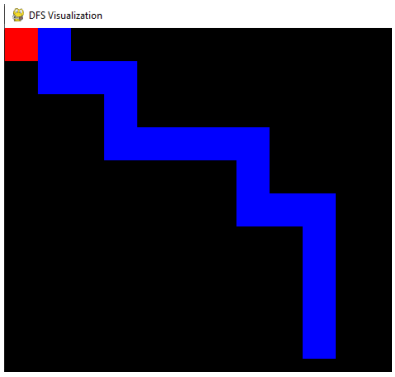


Implementing DFS using Pygame:

```
import pygame
import time

# Initialize Pygame
pygame.init()

# Screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Size of each cell in the grid

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
GREEN = (0, 255, 0)

# Create the screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("DFS Visualization")

# Maze dimensions
ROWS = 10
COLS = 10

# Create a grid-based maze (1 = open, 0 = wall)
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
```

```
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# DFS algorithm
def dfs(maze, start, goal):
    stack = [start]
    visited = set()
    path = {}

    while stack:
        current = stack.pop()
        if current == goal:
            break
        visited.add(current)

        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:  # Right, Down, Left, Up
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            if (
                0 <= next_cell[0] < ROWS
                and 0 <= next_cell[1] < COLS
                and next_cell not in visited
                and maze[next_cell[0]][next_cell[1]] == 1
            ):
                stack.append(next_cell)
                path[next_cell] = current

    # Reconstruct the path
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()
    return full_path

# Draw the maze
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK
            pygame.draw.rect(
                screen,
                color,
                (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )
            pygame.draw.rect(
                screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1
            )

    # Draw the path
    for cell in path:
```

```
            pygame.draw.rect(
                screen,
                BLUE,
                (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )

        # Draw the player
        pygame.draw.rect(
            screen,
            RED,
            (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
        )

# Main function
def main():
    start = (0, 0)
    goal = (9, 9)
    path = dfs(maze, start, goal)
    player_pos = start

    run = True
    clock = pygame.time.Clock()

    while run:
        screen.fill(BLACK)
        draw_maze(maze, path, player_pos)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w:  # Move up
                    next_pos = (player_pos[0] - 1, player_pos[1])
                elif event.key == pygame.K_s:  # Move down
                    next_pos = (player_pos[0] + 1, player_pos[1])
                elif event.key == pygame.K_a:  # Move left
                    next_pos = (player_pos[0], player_pos[1] - 1)
                elif event.key == pygame.K_d:  # Move right
                    next_pos = (player_pos[0], player_pos[1] + 1)
                else:
                    next_pos = player_pos

                # Check if the move is valid
                if (
                    0 <= next_pos[0] < ROWS
                    and 0 <= next_pos[1] < COLS
                    and maze[next_pos[0]][next_pos[1]] == 1
                ):
                    player_pos = next_pos

        pygame.display.update()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```

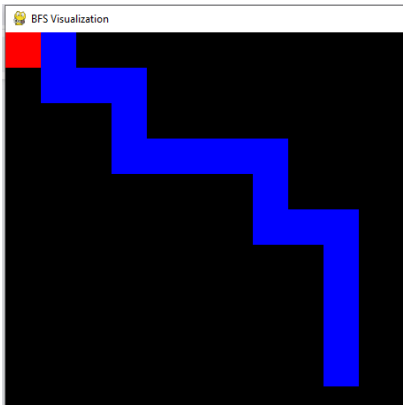


Implementing BFS using Pygame:

```
import pygame
from collections import deque

# Initialize Pygame
pygame.init()

# Screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Size of each cell in the grid

# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
GREEN = (0, 255, 0)

# Create the screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("BFS Visualization")

# Maze dimensions
ROWS = 10
COLS = 10

# Create a grid-based maze (1 = open, 0 = wall)
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# BFS algorithm
def bfs(maze, start, goal):
    queue = deque([start])
    visited = set()
```

```
    visited.add(start)
    path = {}

    while queue:
        current = queue.popleft()
        if current == goal:
            break

        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:  # Right, Down, Left, Up
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            if (
                0 <= next_cell[0] < ROWS
                and 0 <= next_cell[1] < COLS
                and next_cell not in visited
                and maze[next_cell[0]][next_cell[1]] == 1
            ):
                queue.append(next_cell)
                visited.add(next_cell)
                path[next_cell] = current

    # Reconstruct the path
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()
    return full_path

# Draw the maze
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK
            pygame.draw.rect(
                screen,
                color,
                (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE),
            )
            pygame.draw.rect(
                screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1
            )

    # Draw the path
    for cell in path:
        pygame.draw.rect(
            screen,
            BLUE,
            (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
        )

    # Draw the player
    pygame.draw.rect(
        screen,
        RED,
        (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE),
    )
```

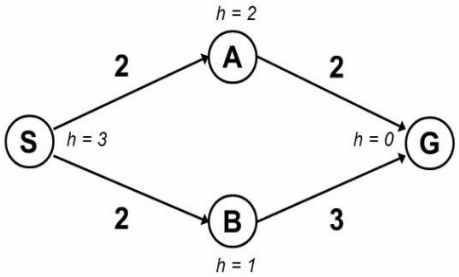
```
# Main function
def main():
    start = (0, 0)
    goal = (9, 9)
    path = bfs(maze, start, goal)
    player_pos = start

    run = True
    clock = pygame.time.Clock()

    while run:
        screen.fill(BLACK)
        draw_maze(maze, path, player_pos)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w:  # Move up
                    next_pos = (player_pos[0] - 1, player_pos[1])
                elif event.key == pygame.K_s:  # Move down
                    next_pos = (player_pos[0] + 1, player_pos[1])
                elif event.key == pygame.K_a:  # Move left
                    next_pos = (player_pos[0], player_pos[1] - 1)
                elif event.key == pygame.K_d:  # Move right
                    next_pos = (player_pos[0], player_pos[1] + 1)
                else:
                    next_pos = player_pos

                # Check if the move is valid
                if (
                    0 <= next_pos[0] < ROWS
                    and 0 <= next_pos[1] < COLS
                    and maze[next_pos[0]][next_pos[1]] == 1
                ):
                    player_pos = next_pos

        pygame.display.update()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()
```



Lab #06

Activity- 1:

Implement A* search in the following state graph, find the shortest path between node S to Node G. Also solve it manually.




```
from collections import deque
import heapq

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list (queue)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.popleft()  # Dequeue the first node
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list (stack)
    closed_list = []  # Closed list (visited nodes)

    while open_list:
        node = open_list.pop()  # Pop the last node (LIFO)
        closed_list.append(node)  # Add it to the closed list

        # Check if the goal is reached
        if node == goal:
            return open_list, closed_list

        # Add neighbors to the open list if not already visited
        for neighbor in reversed(graph[node]):  # Reverse to maintain DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list  # Return the lists if goal is not found

def a_star_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start))  # (f(n), node)
```

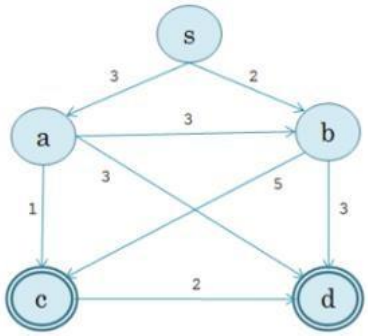
```
    closed_list = []  # Closed list (visited nodes)
    g_costs = {start: 0}  # Cost from start to each node
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure based on the image
graph_bfs_dfs = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# Graph structure with edge costs
graph_a_star = {
    'S': [('A', 2), ('B', 2)],
    'A': [('G', 2)],
    'B': [('G', 3)],
    'G': []
}

# Heuristic values for each node
heuristic = {
    'S': 3,
    'A': 2,
    'B': 1,
```

```
    'G': 0
}

# Initial and goal states for BFS and DFS
start_node_bfs_dfs = 'A'
goal_node_bfs_dfs = 'K'

# Perform BFS
open_list, closed_list = bfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)

# Display results
print("BFS Open List:", list(open_list))
print("BFS Closed List:", closed_list)

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)

# Display results
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)

# Initial and goal states for A* Search
start_node_a_star = 'S'
goal_node_a_star = 'G'

# Perform A* Search
path, closed_list = a_star_search(graph_a_star, heuristic, start_node_a_star, goal_node_a_star)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['S', 'A', 'G']
Closed List: ['S', 'B', 'A', 'G']
```

Activity- 2:

Implement A* search in the following state graph, find the shortest path between node ‘s’ to node ‘c’ or ‘d’. Also solve it manually.



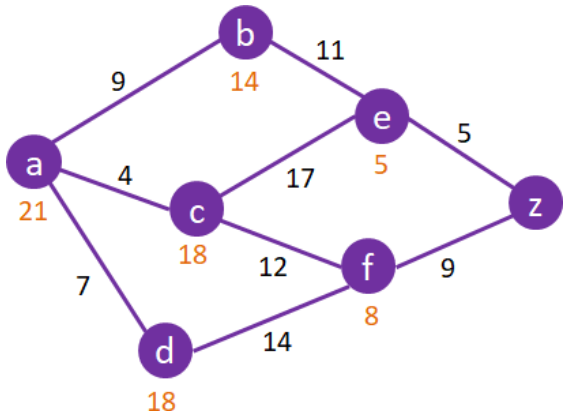
h(s)	h(a)	h(b)	h(c)	h(d)
1	3	3	0	0

```
import heapq

def a_star_search(graph, heuristic, start, goals):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start))  # (f(n), node)
    closed_list = []  # Closed list (visited nodes)
    g_costs = {start: 0}  # Cost from start to each node
```

```
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current in goals:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure with edge costs
graph = {
    's': [('a', 3), ('b', 2)],
    'a': [('c', 3), ('d', 5)],
    'b': [('a', 3), ('d', 3)],
    'c': [],
    'd': []
}

# Heuristic values for each node
heuristic = {
    's': 1,
    'a': 3,
    'b': 3,
    'c': 0,
    'd': 0
}

# Initial and goal states
start_node = 's'
goal_nodes = ['c', 'd']  # Multiple goals

# Perform A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_nodes)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['s', 'b', 'd']
Closed List: ['s', 'b', 'd']
```

Activity- 3:
Implement A* search in the following state graph, find the shortest path between node 'a' to node 'z'. Also solve it manually.

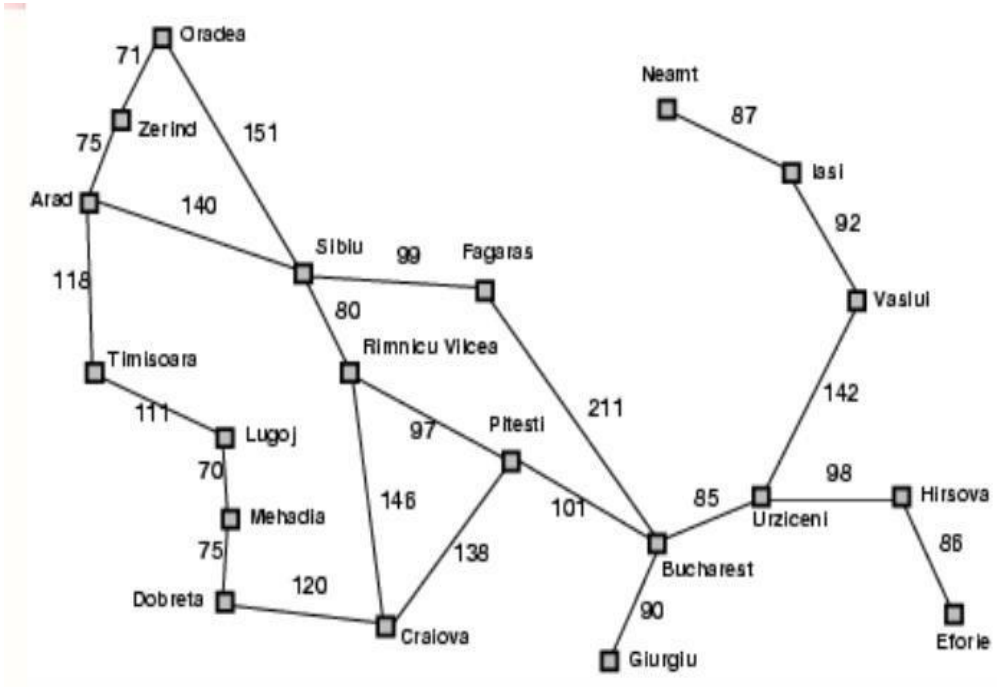



```
import heapq

def a_star_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start))  # (f(n), node)
    closed_list = []  # Closed list (visited nodes)
    g_costs = {start: 0}  # Cost from start to each node
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
```

```
# Graph structure with edge costs
graph = {
    'a': [('b', 9), ('c', 4), ('d', 7)],
    'b': [('e', 11)],
    'c': [('b', 17), ('f', 12)],
    'd': [('c', 14), ('f', 14)],
    'e': [('z', 5)],
    'f': [('e', 8), ('z', 9)],
    'z': []
}

# Heuristic values for each node
heuristic = {
    'a': 21,
    'b': 14,
    'c': 18,
    'd': 18,
    'e': 5,
    'f': 8,
    'z': 0
}

# Initial and goal states
start_node = 'a'
goal_node = 'z'

# Perform A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

Activity- 4:

Implement A* search in the following state graph, find the shortest path between node Arad to Node Bucharest. Also solve it manually.



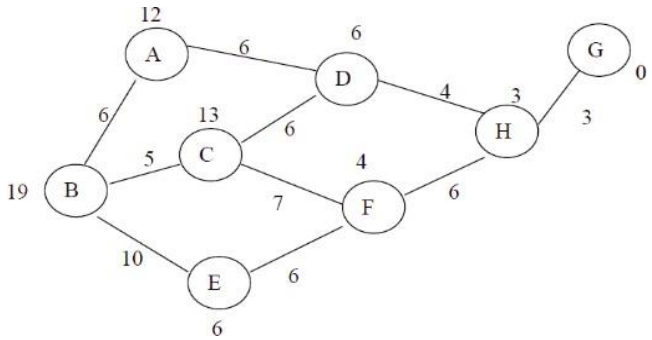
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

```
import heapq

def a_star_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (0 + heuristic[start], start))  # (f(n), node)
    closed_list = []  # Closed list (visited nodes)
    g_costs = {start: 0}  # Cost from start to each node
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest f(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure with edge costs
graph = {
    'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101), ('Giurgiu', 90), ('Urziceni', 85)],
    'Giurgiu': [('Bucharest', 90)],
    'Urziceni': [('Bucharest', 85), ('Hirsova', 98), ('Vaslui', 142)],
    'Hirsova': [('Urziceni', 98), ('Eforie', 86)],
    'Eforie': [('Hirsova', 86)],
    'Vaslui': [('Urziceni', 142), ('Iasi', 92)],
    'Iasi': [('Vaslui', 92), ('Neamt', 87)],
```

```
    'Neamt': [('Iasi', 87)]
}

# Heuristic values for each node (straight-line distance to Bucharest)
heuristic = {
    'Arad': 366,
    'Bucharest': 0,
    'Craiova': 160,
    'Drobeta': 242,
    'Eforie': 161,
    'Fagaras': 176,
    'Giurgiu': 77,
    'Hirsova': 151,
    'Iasi': 226,
    'Lugoj': 244,
    'Mehadia': 241,
    'Neamt': 234,
    'Oradea': 380,
    'Pitesti': 100,
    'Rimnicu Vilcea': 193,
    'Sibiu': 253,
    'Timisoara': 329,
    'Urziceni': 80,
    'Vaslui': 199,
    'Zerind': 374
}

# Initial and goal states
start_node = 'Arad'
goal_node = 'Bucharest'

# Perform A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

Activity- 5:

Implement Greedy Best First Search in the following state graph, find the shortest path between node 'A' to Node 'G'. Also solve it manually.



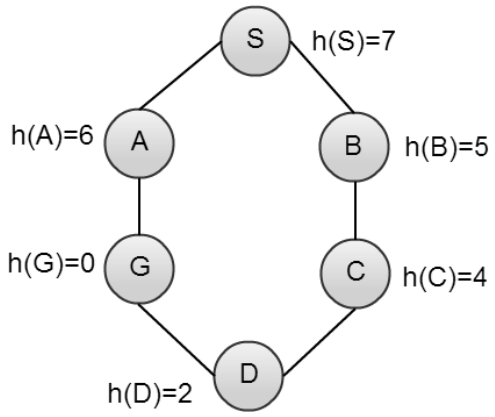
```
import heapq

def greedy_best_first_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (heuristic[start], start))  # (h(n), node)
    closed_list = []  # Closed list (visited nodes)
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest h(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            if neighbor not in closed_list:
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure with edge costs
graph = {
    'A': [('B', 6), ('D', 6)],
    'B': [('A', 6), ('C', 5), ('E', 10)],
    'C': [('B', 5), ('D', 13), ('F', 7)],
    'D': [('A', 6), ('C', 13), ('H', 4)],
    'E': [('B', 10), ('F', 6)],
    'F': [('C', 7), ('E', 6), ('H', 4)],
    'H': [('D', 4), ('F', 6), ('G', 3)],
    'G': [('H', 3)]
}

# Heuristic values for each node
heuristic = {
    'A': 12,
    'B': 19,
    'C': 13,
    'D': 6,
    'E': 10,
    'F': 7,
    'H': 3,
    'G': 0
}

# Initial and goal states
```

```
start_node = 'A'
goal_node = 'G'

# Perform Greedy Best-First Search
path, closed_list = greedy_best_first_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['A', 'D', 'H', 'G']
Closed List: ['A', 'D', 'H', 'G']
```

Activity- 6:

Implement Best First Search in the following state graph, find the shortest path between node 'S' to Node 'G'. Also solve it manually.




```
import heapq

def best_first_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for open list
    heapq.heappush(open_list, (heuristic[start], start))  # (h(n), node)
    closed_list = []  # Closed list (visited nodes)
    came_from = {}  # To reconstruct the path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get the node with the lowest h(n)
        closed_list.append(current)

        # Check if the goal is reached
        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        # Explore neighbors
        for neighbor, cost in graph[current]:
            if neighbor not in closed_list:
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))
                came_from[neighbor] = current

    return None, closed_list  # Return None if no path is found

def reconstruct_path(came_from, current):
    path = [current]
```

```
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph structure with edge costs
graph = {
    'S': [('A', 1), ('B', 1)],
    'A': [('S', 1), ('G', 1)],
    'B': [('S', 1), ('C', 1)],
    'C': [('B', 1), ('D', 1)],
    'D': [('C', 1), ('G', 1)],
    'G': [('A', 1), ('D', 1)]
}

# Heuristic values for each node
heuristic = {
    'S': 7,
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 2,
    'G': 0
}

# Initial and goal states
start_node = 'S'
goal_node = 'G'

# Perform Best-First Search
path, closed_list = best_first_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Shortest Path: ['S', 'B', 'C', 'D', 'G']
Closed List: ['S', 'B', 'C', 'D', 'G']
```

Lab #08

Activity- 1:

Convert following sentences in to PROLOG FACTS and specify whether they represent RELATIONSHIP or PROPERTY:

Declarative Sentence	PROLOG FACT	RELATIONSHIP or PROPERTY
SSUET is university	university(ssuet).	% Property
Faheem likes ice-cream	likes(faheem, icecream).	% Relationship
Birds can fly	can_fly(bird).	% Property
Switch is off	off(switch).	% Property
Sara is nephew of Saima	nephew(sara, saima).	% Relationship

lab08 - Notepad

File Edit Format View Help

```
% Activity 1 Facts
university(ssuet).        % Property
likes(faheem, icecream).   % Relationship
can_fly(bird).             % Property
off(switch).               % Property
nephew(sara, saima).        % Relationship
```

GNU Prolog console

File Edit Terminal Prolog Help

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- change_directory('D:/Sir Syed University/6th Semester/AI Lab').

yes
| ?- [lab08].
compiling D:/Sir Syed University/6th Semester/AI Lab/lab08.pl for byte code...
D:/Sir Syed University/6th Semester/AI Lab/lab08.pl compiled, 8 lines read - 832 bytes written, 6 ms

yes
| ?- university(ssuet).

yes
| ?- likes(faheem, icecream).

yes
| ?- can_fly(bird).

yes
| ?- off(switch).

yes
| ?- nephew(sara, saima).

yes
| ?- nephew(sara, x).

no
| ?- off(x).

no
| ?- nephew(sara, saima).

yes
| ?- nephew(sara, X).

X = saima

yes
```

Activity- 2:

Make prolog facts from following sentences and answer the questions by typing queries on prolog.

- **Saleem is father of Aslam.**
- **Dania is mother of Aslam.**
- **Aslam and Saeed are brothers.**
- **Faiza is sister of Ali.**
- **Sana is enemy of Sumra.**

```
% Activity 2 Facts
father(saleem, aslam).
mother(dania, aslam).
brother(aslam, saeed).
brother(saeed, aslam).
sister(faiza, ali).
enemy(sana, sumra).
```

Questions

1. List brothers.

```
yes
| ?- brother(X, Y).

X = aslam
Y = saeed ?

(15 ms) yes
```

2. Who is father of Aslam?

```
| ?- father(X, aslam).

X = saleem

yes
```

3. Dania is mother of whom?

```
| ?- mother(dania, X).

X = aslam

yes
```

4. Who is brother of Aslam?

```
| ?- brother(X, aslam).

X = saeed

yes
```

5. Faiza is sister of whom?

```
| ?- sister(faiza, X).

X = ali

yes
```

Activity- 3:

Create a file and write below fact and execute this using below question.

```
has(jack,apples).
has(ann,plums).
has(dan,money).
fruit(apples).
fruit(plums).
```

Questions

1. **List fruits.**

```
| ?- fruit(X).

X = apple ?

yes
```

2. **Which fruit does Jack have?**

```
| ?- has(jack, X).

X = apple ?

yes
```

3. **Does Dan have something?**

```
| ?- has(dan, X).

X = mango

yes
```

4. **Who has apples and who has plums?**

```
| ?- has(X, apple).

X = jack ?

yes
| ?- has(X, plum).

X = jack ?

yes
```

5. **Does someone have apples and plums?**

```
| ?- has(X, apple), has(X, plum).

X = jack ?

yes
```

6. **Does Dan have fruits?**

```
| ?- has(dan, X).

X = mango

yes
```

Activity- 4:

Write down the PROLOG Rules for the following sentences:

Declarative Sentences	PROLOG RULES
Person1 is sibling of Person2 if Person3 is parent of Person1 and Person3 is parent of Person2.	sibling(Person1, Person2) :- parent(Person3, Person1), parent(Person3, Person2), Person1 \= Person2.
Person1 is child of Person2 if Person2 is parent of Person1.	child(Person1, Person2) :- parent(Person2, Person1).
It will rain today if clouds are heavy and air pressure is low.	rain_today :- heavy_clouds, low_air_pressure.

Activity- 5:

A person gets good grades if he studies well and is regular in class. Write program in prolog to check whether a given person gets good grade or not?

```
% Activity 5 - Good Grades
studies_well(faheem).
studies_well(sara).
studies_well(ali).

regular_in_class(faheem).
regular_in_class(ali).

good_grades(Person) :-
    studies_well(Person),
    regular_in_class(Person).
```

```
| ?- good_grades(faheem).

yes
| ?- good_grades(sarah).

no
| ?- good_grades(ali).

no
```

Activity- 6:

Here are some declarative sentences convert them into PROLOG facts and answer following questions.

- **Kashif likes music.**
- **Kashif likes cricket.**
- **Saba likes chips.**
- **Saba likes mobile.**
- **Kashif hates cold drinks.**
- **Saba hates driving.**
- **Saba hates parties.**

```
% Activity 6 Facts
likes(kashif, music).
likes(kashif, cricket).
likes(saba, chips).
likes(saba, mobile).

hates(kashif, cold_drinks).
hates(saba, driving).
hates(saba, parties).
```

Questions

1. **Does Saba like chips?**

```
| ?- likes(saba, chips).

true ?

(16 ms) yes
```

2. **Does Kashif hate cricket?**

```
| ?- hates(kashif, cricket).

no
```

3. **Does Kashif like music?**

```
| ?- likes(kashif, music).

true ?

yes
```

4. **What does Kashif like?**

```
| ?- likes(kashif, What).

What = music ?

yes
```

5. **What does Saba like?**

```
| ?- likes(saba, What).

What = chips ?

yes
```

6. **Who likes music?**

```
| ?- likes(Who, music).

Who = kashif ?

yes
```

Activity- 7:

There is a cricket team of 5 players; capabilities of each of theplayer are given in following table.

Player Name	Batsman	Bowler	Fielder
Salman	✓		✓
Afridi	✓	✓	✓
Malik	✓	✓	✓
Shoaib		✓	✓
Tanveer		✓	

Write the facts from the above table and create the rule for all Rounder and run following goals in PROLOG.

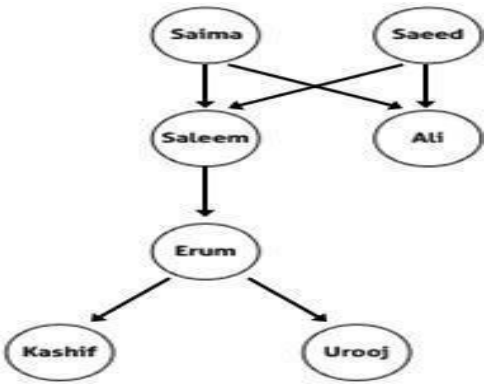
1. **Is Afridi all-rounder?**
2. **Is Shoaib all-rounder?**
3. **Who are all-rounders in the team?**

```
% Activity 7 - Cricket Team Capabilities
batsman(salman).
batsman(afridi).
batsman(malik).
batsman(shoaib).

bowler(salman).
bowler(afridi).
bowler(malik).
bowler(shoaib).
bowler(tanveer).

fielder(salman).
fielder(afridi).
fielder(shoaib).

% Rule: All-rounder is a player who is a batsman, bowler, and fielder
all_rounder(Player) :-
    batsman(Player),
    bowler(Player),
    fielder(Player).
```

```
| ?- all_rounder(afridi).

yes
| ?- all_rounder(shoaib).

yes
| ?- all_rounder(X).

X = salman ?

(16 ms) yes
```

Activity- 8:

Case Study: Family Tree: Consider the following family tree shown in figure below



```
% Activity 8 Facts
father(saeed, saleem).  % Saeed is the father of Saleem
father(saeed, ali).     % Saeed is the father of Ali
father(saleem, erum).   % Saleem is the father of Erum

mother(saima, saleem).  % Saima is the mother of Saleem
mother(saima, ali).     % Saima is the mother of Ali
mother(erum, kashif).   % Erum is the mother of Kashif
mother(erum, urooj).    % Erum is the mother of Urooj

% Rules
% Grandfather: X is grandfather of Y if X is father of Z and Z is parent of Y
grandfather(X, Y) :-
    father(X, Z),
    (father(Z, Y); mother(Z, Y)).

% Grandmother: X is grandmother of Y if X is mother of Z and Z is parent of Y
grandmother(X, Y) :-
    mother(X, Z),
    (father(Z, Y); mother(Z, Y)).

% Siblings: X and Y have same father and mother but are not the same person
brother(X, Y) :-
    father(F, X), father(F, Y),
    mother(M, X), mother(M, Y),
    X \= Y.

sister(X, Y) :-
    father(F, X), father(F, Y),
    mother(M, X), mother(M, Y),
    X \= Y.

% Direct parent relationships
father_of(X, Y) :- father(X, Y).
mother_of(X, Y) :- mother(X, Y).
```

Questions:

1. **Is Saleem father of Ali?**

```
| ?- father(saleem, ali).

no
```

2. **Is Kashif brother of Urooj?**

```
| ?- brother(kashif, urooj).

no
```

3. **Who is father of Kashif?**

```
| ?- father(X, kashif).

no
```

4. **Is Erum mother of Urooj?**

```
| ?- mother(erum, urooj).

yes
```

5. Who is grandfather of Urooj?

```
| ?- grandfather(X, urooj).

X = saleem
```

6. Is Salma grandmother of Erum?

```
| ?- grandmother(salma, erum).

no
```

7. Saeed is father of whom?

```
| ?- father(saeed, X).

X = saleem ? ;

X = ali

yes
```

8. Salma is mother of whom?

```
| ?- mother(salma, X).

no
```

Activity- 9:

Create selection criteria using rules.

1. **starter(soup).**
2. **starter(salad).**
3. **main_course(rice).**
4. **main_course(pizza).**
5. **main_course(burger).**
6. **desert(custard).**
7. **desert(ice_cream).**

Create rules for hungry, very_hungry and on_diet.

- **For hungry: only starter and main course**
- **For very_hungry starter, main course and desert**
- **For on_diet it displays only starters.**

```
% Activity 9
% Starters
starter(soup).
starter(salad).

% Main Course
main_course(rice).
main_course(pizza).
main_course(burger).

% Desert
desert(custard).
desert(ice_cream).

% Hungry: starter and main course
hungry(Item) :-
    starter(Item);
    main_course(Item).

% Very Hungry: starter, main course and desert
very_hungry(Item) :-
    starter(Item);
    main_course(Item);
    desert(Item).
```

```
% On Diet: only starter
on_diet(Item) :-
    starter(Item).
| ?- hungry(X).

X = soup ? ;

X = salad ? ;

X = rice ? ;

X = pizza ? ;

X = burger

yes
| ?- very_hungry(X).

X = soup ? ;

X = salad ? ;

X = rice ? ;

X = pizza ? ;

X = burger ? ;

X = custard ? ;

X = ice_cream

(47 ms) yes
| ?-  on_diet(X).

X = soup ? ;

X = salad

yes
```

Activity- 10:
Write a program that doubles any number that is taken from user as input.

```
% Activity 10
double_number :-
    write('Enter a number: '),
    read(Number),
    Double is Number * 2,
    write('Double is: '),
    write(Double), nl.

| ?- double_number.
Enter a number: 7.
Double is: 14

yes
```

Activity- 11:
Calculate Arithmetic mean of any three numbers.

```
% Activity 11
arithmetic_mean :-
    write('Enter first number: '),
    read(A),
    write('Enter second number: '),
    read(B),
    write('Enter third number: '),
    read(C),
    Mean is (A + B + C) / 3,
    write('Arithmetic Mean is: '),
    write(Mean), nl.
```

```
| ?- arithmetic_mean.
Enter first number: 5.
Enter second number: 10.
Enter third number: 15.
Arithmetic Mean is: 10.0
```

Activity- 12:

Create a basic calculator prolog program for making basic calculations (add, subtract, multiply, divide, average). Use separate goal statements for making each calculation (with constant numeric values).

```
% Activity 12
% Addition
add :-
   write('Enter first number: '), read(A),
   write('Enter second number: '), read(B),
   Result is A + B,
   write('Addition: '), write(Result), nl.

% Subtraction
subtract :-
   write('Enter first number: '), read(A),
   write('Enter second number: '), read(B),
   Result is A - B,
   write('Subtraction: '), write(Result), nl.

% Multiplication
multiply :-
   write('Enter first number: '), read(A),
   write('Enter second number: '), read(B),
   Result is A * B,
   write('Multiplication: '), write(Result), nl.

% Division
divide :-
   write('Enter numerator: '), read(A),
   write('Enter denominator: '), read(B),
   (   B =\= 0
   ->  Result is A / B,
      write('Division: '), write(Result), nl
   ;   write('Error: Division by zero is not allowed.'), nl
   ).

% Average of three numbers
average :-
   write('Enter first number: '), read(A),
   write('Enter second number: '), read(B),
   write('Enter third number: '), read(C),
```

```
  Avg is (A + B + C) / 3,
  write('Average: '), write(Avg), nl.
```

```
| ?- add.
Enter first number: 2.
Enter second number: 3.
Addition: 5

(16 ms) yes
| ?- subtract.
Enter first number: 3.
Enter second number: 1.
Subtraction: 2

yes
| ?- multiply.
Enter first number: 3.
Enter second number: 3.
Multiplication: 9

(15 ms) yes
| ?- divide.
Enter numerator: 2.
Enter denominator: 6.
Division: 0.33333333333333331

(15 ms) yes
| ?- average.
Enter first number: 2.
Enter second number: 3.
Enter third number: 4.
Average: 3.0

yes
```

Activity- 13:

Make a calculator program by using write() and read() predicates, for taking numeric input from user and displaying text after program execution.

```
% Activity 13
calculator :-
    write('Enter the first number: '), read(A),
    write('Enter the second number: '), read(B),

    Sum is A + B,
    Diff is A - B,
    Prod is A * B,

    (   B =\= 0
    ->  Div is A / B
    ;   Div = 'undefined (division by zero)'
    ),

    Avg is (A + B) / 2,

    nl, write('--- Calculation Results ---'), nl,
    write('Addition: '), write(Sum), nl,
    write('Subtraction: '), write(Diff), nl,
    write('Multiplication: '), write(Prod), nl,
    write('Division: '), write(Div), nl,
    write('Average: '), write(Avg), nl.
```

```
| ?- calculator.
Enter the first number: 2.
Enter the second number: 1.

--- Calculation Results ---
Addition: 3
Subtraction: 1
Multiplication: 2
Division: 2.0
Average: 1.5

yes
```

Lab #08

Activity- 1: Write a Python program to convert a list of numeric value into a one-dimensional NumPy array.

```
# Activity 1
list1 = [10, 20, 30, 40]
array1 = np.array(list1)
print("Activity 1:", array1)
```

```
Activity 1: [10 20 30 40]
```

Activity- 2: Create a 3x3 matrix with values ranging from 2 to 11.

```
# Activity 2
matrix_3x3 = np.arange(2, 11).reshape(3, 3)
print("\nActivity 2:\n", matrix_3x3)
```

```
Activity 2:
 [[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
```

Activity- 3: Create a null vector of size 15 and update fifth value to 12 and ninth value to 17.

```
# Activity 3
null_vector = np.zeros(15)
null_vector[4] = 12
null_vector[8] = 17
print("\nActivity 3:", null_vector)
```

```
Activity 3: [ 0.  0.  0.  0. 12.  0.  0.  0. 17.  0.  0.  0.  0.  0.  0.]
```

Activity- 4: Take 2D and 3D array and perform transpose function.

```
# Activity 4
array_2d = np.array([[1, 2], [3, 4]])
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print("\nActivity 4:")
print("2D Transpose:\n", array_2d.T)
print("3D Transpose:\n", np.transpose(array_3d, (1, 0, 2)))
```

```
Activity 4:
2D Transpose:
 [[1 3]
 [2 4]]
3D Transpose:
 [[[1 2]
  [5 6]]

 [[3 4]
  [7 8]]]
```

Activity- 5: Write a Python program to find the real and imaginary parts of an array of complex numbers.

```
# Activity 5
complex_array = np.array([1+2j, 3+4j, 5+6j])
print("\nActivity 5:")
print("Real parts:", complex_array.real)
print("Imaginary parts:", complex_array.imag)
```

```
Activity 5:
Real parts: [1. 3. 5.]
Imaginary parts: [2. 4. 6.]
```

Activity- 6: Multiply a 5x3 matrix by a 3x2 matrix create a real matrix product using dot ( ).

```
# Activity 6
matrix_a = np.random.rand(5, 3)
matrix_b = np.random.rand(3, 2)
product = np.dot(matrix_a, matrix_b)
print("\nActivity 6:\n", product)
```

```
Activity 6:
 [[0.06447387 0.36992028]
 [0.59269926 0.91108057]
 [0.23539423 0.60753601]
 [0.8514891  1.48429859]
 [0.48487478 0.77945942]]
```

Activity- 7: Create array by 2x3 and perform following function:

- **Number of dimensions**
- **Total number of element/size in an array**
- **What type of array stores in memory(dtype)**

```
# Activity 7
array_2x3 = np.array([[1, 2, 3], [4, 5, 6]])
print("\nActivity 7:")
print("Dimensions:", array_2x3.ndim)
print("Total elements:", array_2x3.size)
print("Data type:", array_2x3.dtype)
```

```
Activity 7:
Dimensions: 2
Total elements: 6
Data type: int32
```

Activity- 8: Write a Python program to create a 2-D array with reshape (5, 5) and filled 1 on the border and 0 inside using slicing method.

```
# Activity 8
array_5x5 = np.ones((5, 5))
array_5x5[1:-1, 1:-1] = 0
print("\nActivity 8:\n", array_5x5)
```

```
Activity 8:
 [[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

Activity- 9: Write a Python program to create a 3-D array with ones on the diagonal and zeros elsewhere.

```
# Activity 9
array_3d_diag = np.eye(3)
array_3d = np.zeros((3, 3, 3))
for i in range(3):
    array_3d[i][i][i] = 1
print("\nActivity 9:\n", array_3d)
```

```
Activity 9:
 [[[1. 0. 0.]
  [0. 0. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 1. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 0. 0.]
  [0. 0. 1.]]]
```

Activity- 10: Write a Python program to find common values between two arrays using intersect1d ( ). Take array1= [1 30 40 60 80, 90] and array2= [50,10,30,40] as input.

```
# Activity 10
arr1 = np.array([1, 30, 40, 60, 80, 90])
arr2 = np.array([50, 10, 30, 40])
common = np.intersect1d(arr1, arr2)
print("\nActivity 10:", common)
```

```
Activity 10: [30 40]
```

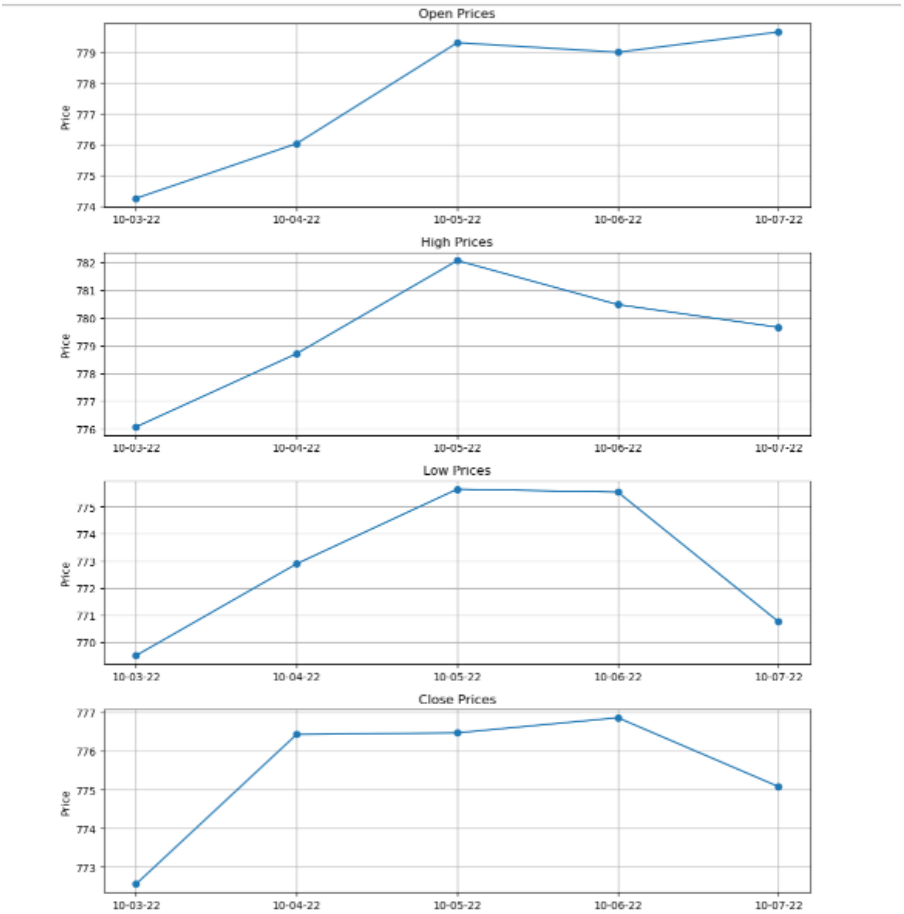
Table A-11: Financial data of a company

Date	Open	High	Low	Close
10-03-22	774.25	776.065002	769.5	772.559998
10-04-22	776.030029	778.710022	772.890015	776.429993
10-05-22	779.309998	782.070007	775.650024	776.469971
10-06-22	779	780.47998	775.539978	776.859985
10-07-22	779.659973	779.659973	770.75	775.080017

```
import pandas as pd
import matplotlib.pyplot as plt

# Financial data
data = {
    "Date": ["10-03-22", "10-04-22", "10-05-22", "10-06-22", "10-07-22"],
    "Open": [774.25, 776.030029, 779.309998, 779, 779.659973],
    "High": [776.065002, 778.710022, 782.070007, 780.47998, 779.659973],
    "Low": [769.5, 772.890015, 775.650024, 775.539978, 770.75],
    "Close": [772.559998, 776.429993, 776.469971, 776.859985, 775.080017]
}

df = pd.DataFrame(data)
```

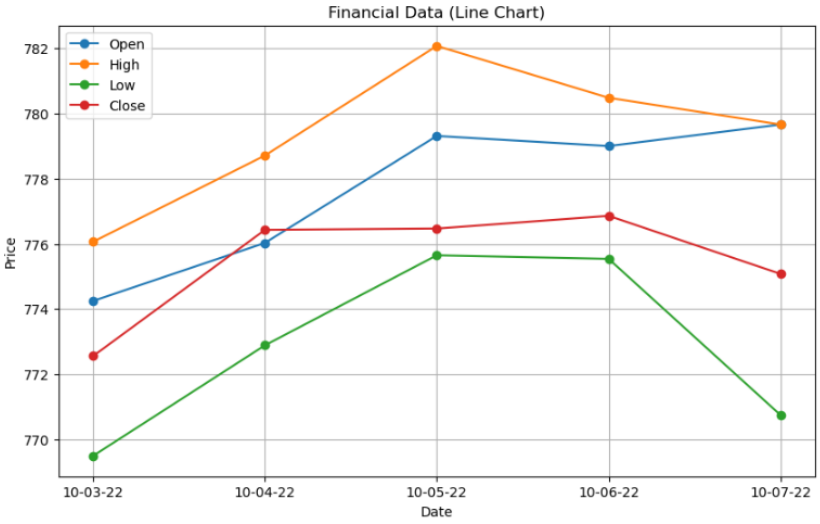
Activity- 11: Write a Python program to draw individual line charts of the financial data of a company given in Table A-11 between October 3, 2022 to October 7, 2022 using subplot for each attribute

```
#Activity 11
fig, axs = plt.subplots(4, 1, figsize=(10, 12))
attributes = ['Open', 'High', 'Low', 'Close']

for i, attr in enumerate(attributes):
    axs[i].plot(df['Date'], df[attr], marker='o')
    axs[i].set_title(f'{attr} Prices')
    axs[i].set_ylabel('Price')
    axs[i].grid(True)

plt.tight_layout()
plt.show()
```



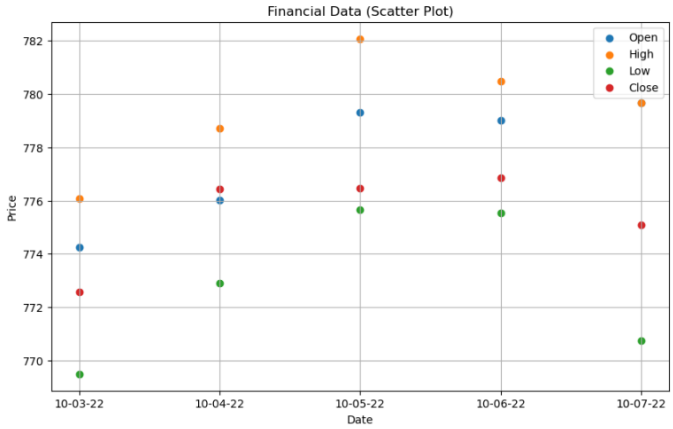
Activity- 12: Write a Python program to draw line charts on the same graph of the financial data of a company given in Table A-11.

```
#Activity 12
plt.figure(figsize=(10, 6))
for attr in attributes:
    plt.plot(df['Date'], df[attr], marker='o', label=attr)

plt.title('Financial Data (Line Chart)')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



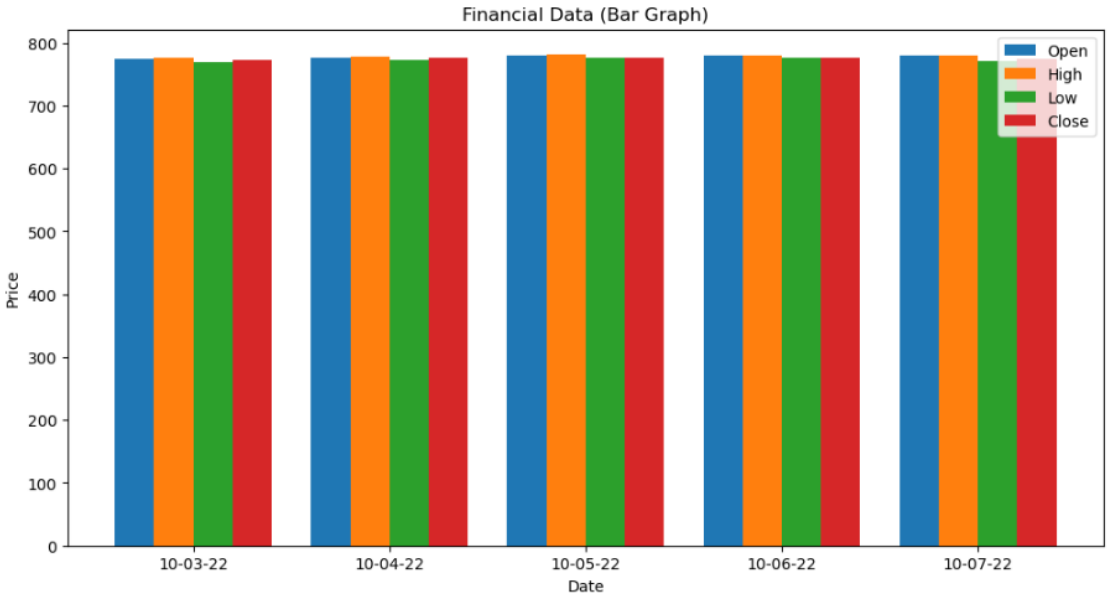
Activity- 13: Write a Python program to draw scatter plot graph of the financial data of a company given in Table A-11.

```
#Activity 13
plt.figure(figsize=(10, 6))
for attr in attributes:
    plt.scatter(df['Date'], df[attr], label=attr)

plt.title('Financial Data (Scatter Plot)')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



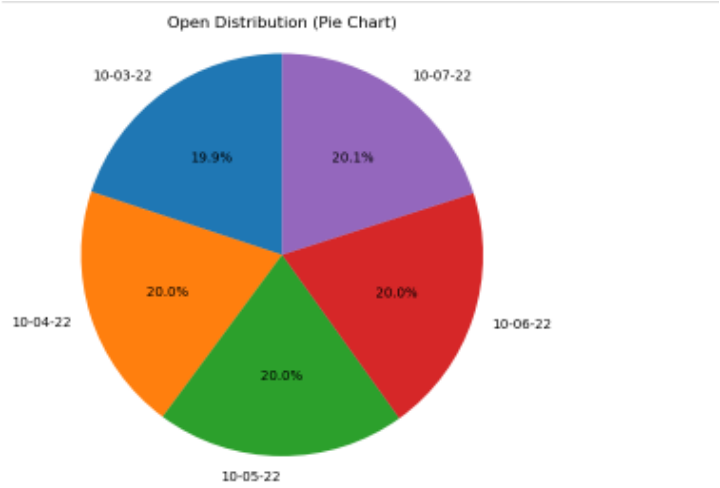
Activity- 14: Write a Python program to draw bar graphs for each attribute of the financial data of a company given in Table A-11.

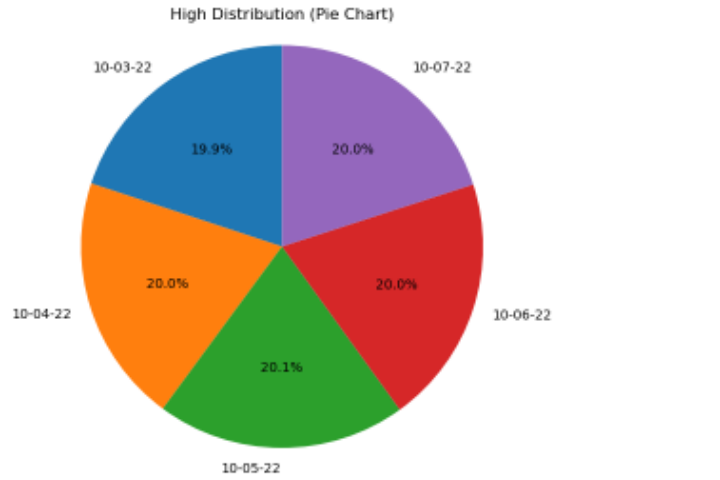
```
#Activity 14
x = df['Date']
bar_width = 0.2
x_indexes = range(len(x))

plt.figure(figsize=(12, 6))
for i, attr in enumerate(attributes):
    plt.bar([p + i*bar_width for p in x_indexes], df[attr], width=bar_width, label=attr)

plt.xticks([p + 1.5*bar_width for p in x_indexes], x)
plt.title('Financial Data (Bar Graph)')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```

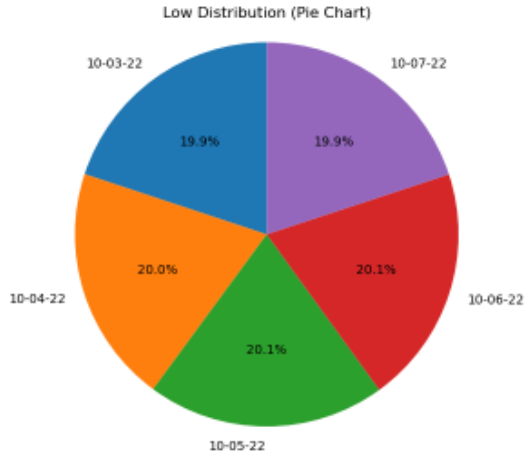


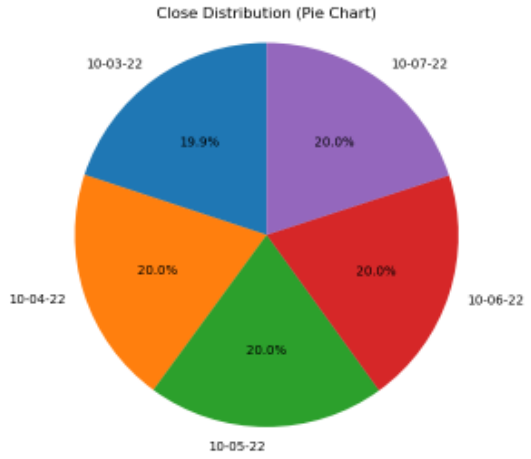
Activity- 15: Write a Python program to draw pie chart for each attribute of the financial data of a company given in Table A-11.

```
#Activity 15
for attr in attributes:
    plt.figure(figsize=(6, 6))
    plt.pie(df[attr], labels=df['Date'], autopct='%1.1f%%', startangle=90)
    plt.title(f'{attr} Distribution (Pie Chart)')
    plt.axis('equal')
    plt.show()
```


Open Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.1%
20.0%
20.0%
10-04-22
10-06-22
20.0%
10-05-22


High Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.0%
20.0%
20.0%
10-04-22
10-06-22
20.1%
10-05-22


Low Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
19.9%
20.0%
20.1%
10-04-22
10-06-22
20.1%
10-05-22


Close Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.0%
20.0%
20.0%
10-04-22
10-06-22
20.0%
10-05-22

Lab #10

Activity- 1: Consider the dataset given below, implement K-Means Clustering algorithm using K=2. Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model and compute it manually.

S. No.	A	B
1	170	56
2	168	60
3	185	72
4	188	77
5	177	76
6	180	71

```
import numpy as np
from sklearn.metrics import accuracy_score

# Step 1: Data
data = np.array([
    [170, 56],
    [168, 60],
    [185, 72],
    [188, 77],
    [177, 76],
    [180, 71]
])

# Step 2: Initial centroids (chosen from data points 0 and 2)
centroid1 = data[0]
centroid2 = data[2]

def euclidean_distance(p1, p2):
    return np.sqrt(np.sum((p1 - p2)**2))

# Step 3: Assign clusters
clusters = []
for point in data:
    dist1 = euclidean_distance(point, centroid1)
    dist2 = euclidean_distance(point, centroid2)
    if dist1 < dist2:
        clusters.append(0)  # Cluster 0
    else:
        clusters.append(1)  # Cluster 1

clusters = np.array(clusters)

# Step 4: Recalculate centroids
cluster1_points = data[clusters == 0]
cluster2_points = data[clusters == 1]

new_centroid1 = np.mean(cluster1_points, axis=0)
new_centroid2 = np.mean(cluster2_points, axis=0)
```

```
# Step 5: Accuracy
# Assuming ground truth: first 2 points belong to cluster 0, rest to cluster 1
true_labels = np.array([0, 0, 1, 1, 1, 1])

# Since cluster labels can be swapped, check both possibilities
accuracy1 = accuracy_score(true_labels, clusters)
accuracy2 = accuracy_score(true_labels, 1 - clusters)

accuracy = max(accuracy1, accuracy2)

# Output
print("Initial Centroid 1:", centroid1)
print("Initial Centroid 2:", centroid2)
print("New Centroid 1:", new_centroid1)
print("New Centroid 2:", new_centroid2)
print("Cluster Assignments:", clusters)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

```
Initial Centroid 1: [170  56]
Initial Centroid 2: [185  72]
New Centroid 1: [169.  58.]
New Centroid 2: [182.5  74. ]
Cluster Assignments: [0 0 1 1 1 1]
Accuracy: 100.00%
```

Activity- 2: A dataset (income.csv) has been provided. Implement K-Means Clustering Algorithm on this dataset using K (number of clusters = 3). Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score
from itertools import permutations

# Load dataset
df = pd.read_csv("income.csv")

# Separate features and label
X = df.drop("income_level", axis=1)
y_true = df["income_level"]
```

```
# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply KMeans with K=3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# Convert cluster centers back to original scale
centroids_scaled = kmeans.cluster_centers_
centroids_original = scaler.inverse_transform(centroids_scaled)
```

```
# Find best accuracy by checking all permutations of cluster labels
def best_accuracy(true_labels, cluster_labels):
    best_acc = 0
    for perm in permutations([0, 1, 2]):
        mapping = {i: perm[i] for i in range(3)}
        mapped = [mapping[c] for c in cluster_labels]
        acc = accuracy_score(true_labels, mapped)
        best_acc = max(best_acc, acc)
    return best_acc
```

```
# Print centroids and accuracy
accuracy = best_accuracy(y_true, clusters)

print("New Centroid Values (Original Scale):")
for i, c in enumerate(centroids_original):
    print(f"Centroid {i+1}: {c}")

print(f"\nAccuracy of K-Means Clustering (K=3): {accuracy * 100:.2f}%")
```

```
New Centroid Values (Original Scale):
Centroid 1: [2.89293240e+01 2.21945982e+05 9.21150132e+00 1.89502371e+02
 3.49692713e-01 3.64833626e+01]
Centroid 2: [4.76376343e+01 1.58929617e+05 1.08202653e+01 2.03101734e+03
 7.23514103e-01 4.39143301e+01]
Centroid 3: [ 4.17788204e+01  1.88251561e+05  1.09982127e+01 -1.90993887e-11
  1.89838472e+03  4.33440572e+01]

Accuracy of K-Means Clustering (K=3): 60.21%
```

Activity- 3: A dataset (ratings_small.csv) has been provided. Implement K-Means Clustering Algorithm on this dataset using K (number of clusters = 5). Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Step 1: Load the dataset
df = pd.read_csv('ratings_small.csv')

# Step 2: Select relevant features
features = df[['userId', 'movieId', 'rating']]

# Step 3: Normalize the features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)
```

```
# Step 4: Apply K-Means Clustering (K=5)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans.fit(features_scaled)
```

```python
# Step 5: Get cluster centroids (and convert back to original scale)
centroids_scaled = kmeans.cluster_centers_
centroids_original = scaler.inverse_transform(centroids_scaled)

# Step 6: Calculate silhouette score (for clustering accuracy)
silhouette = silhouette_score(features_scaled, kmeans.labels_)

# Display results
print("Centroid values (userId, movieId, rating):")
for idx, centroid in enumerate(centroids_original):
    print(f"Cluster {idx}: userId={centroid[0]:.2f}, movieId={centroid[1]:.2f}, rating={centroid[2]:.2f}")

print(f"\nSilhouette Score (clustering accuracy estimate): {silhouette:.3f}")
```

```
Centroid values (userId, movieId, rating):
Cluster 0: userId=355.60, movieId=80865.56, rating=3.54
Cluster 1: userId=498.51, movieId=4183.92, rating=2.43
Cluster 2: userId=162.99, movieId=4223.11, rating=4.23
Cluster 3: userId=507.84, movieId=4137.52, rating=4.25
Cluster 4: userId=161.41, movieId=4471.59, rating=2.44

Silhouette Score (clustering accuracy estimate): 0.365
```

Lab #09

Activity- 1:

Implement the Decision tree algorithm on the data given in the table 9.1 and predict whether the players can play or not when the weather is overcast and the temperature is mild. Also verify results by solving manually.

Table 9.1 Weather Forecast

Weather	Temperature	Play
Sunny	Hot	Yes
Sunny	Hot	Yes
Overcast	Hot	No
Rain	Mild	Yes
Sunny	Cool	No
Overcast	Cool	No
Rain	Mild	Yes
Rain	Mild	Yes
Overcast	Hot	No
Sunny	Hot	No

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Step 1: Dataset
data = {
    'Weather': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Sunny',
                'Overcast', 'Rain', 'Rain', 'Overcast', 'Sunny'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                    'Cool', 'Mild', 'Mild', 'Hot', 'Hot'],
    'Play': ['Yes', 'Yes', 'No', 'Yes', 'No',
             'No', 'Yes', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)

# Step 2: Create separate LabelEncoders for each column
le_weather = LabelEncoder()
le_temp = LabelEncoder()
le_play = LabelEncoder()
```

```
df['Weather_enc'] = le_weather.fit_transform(df['Weather'])      # Sunny=2, Overcast=0, Rain=1
df['Temperature_enc'] = le_temp.fit_transform(df['Temperature'])  # Hot=1, Mild=2, Cool=0
df['Play_enc'] = le_play.fit_transform(df['Play'])               # Yes=1, No=0

# Step 3: Train Decision Tree
features = df[['Weather_enc', 'Temperature_enc']]
labels = df['Play_enc']

clf = DecisionTreeClassifier()
clf.fit(features, labels)

# Step 4: Predict for Overcast & Mild
test_sample = [[le_weather.transform(['Overcast'])[0], le_temp.transform(['Mild'])[0]]]
prediction = clf.predict(test_sample)
predicted_label = le_play.inverse_transform(prediction)

print("Prediction for Overcast & Mild:", predicted_label[0])
```

```
Prediction for Overcast & Mild: No
```

Activity- 2:

Implement the Decision tree algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Step 1: Load the dataset
df = pd.read_csv("Iris.csv")  # Ensure the file is in the same folder or provide full path

# Step 2: Drop unwanted columns if present
if 'Id' in df.columns:
    df = df.drop('Id', axis=1)

# Step 3: Encode the target column 'Species'
le = LabelEncoder()
df['Species_enc'] = le.fit_transform(df['Species'])  # setosa=0, versicolor=1, virginica=2

# Step 4: Train the Decision Tree model
features = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
labels = df['Species_enc']

clf = DecisionTreeClassifier()
clf.fit(features, labels)
```

```
# Step 5: Take user input for prediction
print("Enter the flower measurements:")
sl = float(input("Sepal Length (cm): "))
sw = float(input("Sepal Width (cm): "))
pl = float(input("Petal Length (cm): "))
pw = float(input("Petal Width (cm): "))

new_sample = [[sl, sw, pl, pw]]
prediction = clf.predict(new_sample)
predicted_species = le.inverse_transform(prediction)

print(f"\n✅ Predicted species: {predicted_species[0]}")
```

```
Enter the flower measurements:
Sepal Length (cm):  23
Sepal Width (cm):  2
Petal Length (cm):  4
Petal Width (cm):  55

✅ Predicted species: Iris-virginica
```

Activity- 3:

Implement Naïve Bayes Algorithm on the dataset given in Table 9.2, to predict whether the players can play or not when the Outlook is Rain, Temperature is Cool, Humidity is High and the Wind is Strong. Also verify results by solving manually.

Table 9.2 Golf play chart

Day	Outlook	Temperature	Humidity	Wind	Play golf
D1	Sunny	Hot	High	Weak	Yes
D2	Sunny	Hot	High	Strong	Yes

D3	Overcast	Hot	High	Weak	No
D4	Rain	Mild	High	Weak	Yes
D5	Sunny	Cool	Normal	Weak	No
D6	Overcast	Cool	Normal	Strong	Yes
D7	Rain	Mild	Normal	Strong	No
D8	Rain	Mild	High	Weak	Yes
D9	Overcast	Hot	Normal	Weak	No
D10	Sunny	Hot	Normal	Strong	No

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

# Step 1: Create the dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Overcast', 'Sunny'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Mild', 'Mild', 'Hot', 'Hot'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Strong'],
    'PlayGolf': ['Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)

# Step 2: Encode categorical columns
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])

# Step 3: Split features and labels
features = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]
labels = df['PlayGolf']
```

```python
# Step 4: Train Naive Bayes classifier
model = GaussianNB()
model.fit(features, labels)

# Step 5: Predict new case: Outlook=Rain, Temp=Cool, Humidity=High, Wind=Strong
# First encode inputs manually:
# You may need to re-map based on label encoding used, here:
# ['Cool', 'Hot', 'Mild'] → 0,1,2
# ['High', 'Normal'] → 0,1
# ['Overcast', 'Rain', 'Sunny'] → 0,1,2
# ['Strong', 'Weak'] → 0,1

new_sample = [[1, 0, 0, 0]]  # Rain, Cool, High, Strong
prediction = model.predict(new_sample)

# Decode label (0 = No, 1 = Yes)
result = 'Yes' if prediction[0] == 1 else 'No'
print(f"✅ Prediction: Will the player play golf? → {result}")
```

Activity- 4:

Consider the following dataset given in Table 9.3. Implement Naïve Bayes Algorithm to classify youth/medium/yes/fair. Also verify results by solving manually.

Table 9.3 Student income chart

age	*income*	*student*	*credit_rating*	*Class: buys_computer*
youth	high	no	fair	no
youth	high	no	excellent	no
middle_aged	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle_aged	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle_aged	medium	no	excellent	yes
middle_aged	high	yes	fair	yes
senior	medium	no	excellent	no

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder

# Step 1: Input data
data = {
    'age': ['youth', 'youth', 'middle_aged', 'senior', 'senior', 'senior', 'middle_aged', 'youth', 'youth', 'senior', 'youth', 'middle_aged', 'middle_age
    'income': ['high', 'high', 'high', 'medium', 'low', 'low', 'low', 'medium', 'low', 'medium', 'medium', 'medium', 'high', 'medium'],
    'student': ['no', 'no', 'no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no'],
    'credit_rating': ['fair', 'excellent', 'fair', 'fair', 'fair', 'excellent', 'excellent', 'fair', 'fair', 'fair', 'excellent', 'excellent', 'fair', 'e
    'buys_computer': ['no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'no']
}

df = pd.DataFrame(data)

# Step 2: Label encode each column with a separate encoder
encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le
```

```
# Step 3: Train Naive Bayes
features = df.drop('buys_computer', axis=1)
labels = df['buys_computer']
model = GaussianNB()
model.fit(features, labels)

# Step 4: Predict for youth/medium/yes/fair
# Use the same label encoders to transform input
new_data = [
    encoders['age'].transform(['youth'])[0],
    encoders['income'].transform(['medium'])[0],
    encoders['student'].transform(['yes'])[0],
    encoders['credit_rating'].transform(['fair'])[0]
]

pred = model.predict([new_data])
result = encoders['buys_computer'].inverse_transform(pred)[0]
print("Prediction for youth/medium/yes/fair:", result)
```

```
Prediction for youth/medium/yes/fair: yes
```

Activity- 5:

Implement the Naïve Bayes algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load the dataset
df = pd.read_csv('adult.csv')

# Step 2: Clean the data
df.replace(' ?', pd.NA, inplace=True)
df.dropna(inplace=True)

# Step 3: Rename columns (optional but helpful for consistency)
df.columns = df.columns.str.strip().str.lower().str.replace('-', '.')

# Step 4: Encode categorical columns
encoders = {}
for col in df.select_dtypes(include='object').columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le
# Step 5: Features and target split
X = df.drop('income', axis=1)
y = df['income']

# Step 6: Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 7: Train the model
model = GaussianNB()
model.fit(X_train, y_train)

# Step 8: Evaluate the model
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Step 9: Manual user input
print("\n--- Enter New User Info ---")
user_input = {}
for col in X.columns:
    if col in encoders:
        val = input(f"{col.replace('.', ' ').capitalize()} (e.g., {encoders[col].classes_[0]}): ")
        try:
            user_input[col] = encoders[col].transform([val])[0]
        except:
            user_input[col] = 0  # fallback for unseen values
    else:
        user_input[col] = float(input(f"{col.replace('.', ' ').capitalize()}: "))
```

```python
# Step 10: Prediction
user_df = pd.DataFrame([user_input])[X.columns]
prediction = model.predict(user_df)
predicted_label = encoders['income'].inverse_transform(prediction)[0]

print(f"\nPredicted income category: {predicted_label}")
```

```
--- Enter New User Info ---
Age:  22
Workclass (e.g., ?):  Private
Fnlwgt:  201499
Education (e.g., 10th):  Bachelors
Education num:  2
Marital status (e.g., Divorced):  Never-married
Occupation (e.g., ?):  Tech-support
Relationship (e.g., Husband):  White
Race (e.g., Amer-Indian-Eskimo):  White
Sex (e.g., Female):  Male
Capital gain:  20
Capital loss:  220
Hours per week:  2
Native country (e.g., ?):  Pakistan

Predicted income category: <=50K
```